

IF3170 INTELIGENSI ARTIFISIAL
TUGAS BESAR 1
Pencarian Solusi Penjadwalan Kelas Mingguan
dengan Local Search



Disusun Oleh:

13523003 Dave Daniell Yanni

13523029 Bryan Ho

13523033 Alvin Christopher Santausa

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025

Daftar Isi

Daftar Isi	2
1. Deskripsi Persoalan	3
2. Pembahasan	3
2.1. Pemilihan Objective Function	3
2.2. Penjelasan Implementasi Algoritma Local Search	5
2.3. Hasil eksperimen dan Analisis	36
3. Kesimpulan dan Saran	93
3.1. Kesimpulan	93
3.2. Saran	94
4. Pembagian Tugas	95
5. Lampiran	95
5.1. Referensi	95
5.2. Link Repository	96

1. Deskripsi Persoalan

Tugas Besar 1 mata kuliah IF3170 Inteligensi Artifisial berfokus pada penyelesaian masalah optimasi kompleks dalam dunia nyata, yaitu penjadwalan kelas mingguan. Persoalan ini melibatkan penempatan serangkaian mata kuliah ke dalam slot waktu dan ruangan yang tersedia selama satu minggu, dengan mempertimbangkan berbagai kendala dan variabel.

Tujuan utamanya adalah menghasilkan jadwal yang seoptimal mungkin dengan meminimalkan konflik dan memaksimalkan kepuasan berbagai pihak yang terlibat. Variabel utama dalam persoalan ini meliputi:

- Kelas Mata Kuliah, yang memiliki atribut seperti kode mata kuliah, jumlah mahasiswa terdaftar, dan jumlah SKS (di mana 1 SKS setara dengan 1 jam).
- Waktu, yang direpresentasikan dalam format integer.
- Ruangan, yang memiliki kapasitas atau kuota tertentu.
- Mahasiswa, yang masing-masing memiliki NIM mahasiswa, daftar mata kuliah yang diambil dan tingkat prioritas untuk setiap mata kuliah.

Untuk menyelesaikan persoalan ini, akan diimplementasikan beberapa algoritma local search, yaitu salah satu varian dari algoritma hill-climbing, Simulated Annealing, dan Genetic Algorithm. Solusi atau jadwal akan direpresentasikan sebagai pemetaan setiap pertemuan mata kuliah ke slot waktu dan ruangan tertentu.

Kualitas dari sebuah jadwal dievaluasi menggunakan sebuah objective function. Beberapa kriteria yang menjadi acuan dalam penyusunan objective function antara lain:

- Minimnya jadwal yang bertabrakan untuk setiap mahasiswa.
- Minimnya konflik penggunaan ruangan dan waktu yang sama oleh beberapa mata kuliah, dengan mempertimbangkan bobot prioritas dari mahasiswa yang mengambil mata kuliah tersebut.
- Memastikan jumlah mahasiswa tidak melebihi kuota ruangan yang tersedia.

Melalui tugas besar ini, akan dilakukan perancangan representasi solusi, implementasi algoritma, serta eksperimen untuk menganalisis dan membandingkan performa dari setiap algoritma dalam menemukan solusi optimal untuk masalah penjadwalan kelas mingguan.

2. Pembahasan

2.1. Pemilihan Objective Function

Objective function atau fungsi tujuan adalah komponen krusial dalam algoritma local search yang berfungsi sebagai metrik untuk mengukur kualitas dari sebuah solusi (state). Dalam konteks penjadwalan ini, algoritma akan berusaha untuk

mencari solusi dengan nilai objective function yang paling minimal, yang mengindikasikan jadwal dengan "biaya" atau "penalti" terendah.

Pada implementasi ini, kami merancang tiga variasi objective function yang masing-masing mengukur aspek kualitas jadwal yang berbeda. Pengguna program dapat memilih salah satu dari ketiga fungsi ini untuk digunakan dalam proses pencarian solusi. Berikut adalah penjelasan untuk setiap fungsi.

1. Penalti Konflik Jadwal Mahasiswa

Fungsi ini berfokus pada kualitas jadwal dari sudut pandang mahasiswa. Tujuannya adalah untuk meminimalkan jumlah jadwal kuliah yang tumpang tindih (bentrok) bagi setiap mahasiswa. Jadwal yang ideal adalah jadwal di mana tidak ada satu pun mahasiswa yang memiliki dua atau lebih mata kuliah pada slot waktu yang sama.

- Logika Perhitungan

Penalti dihitung dengan menjumlahkan semua kejadian konflik jadwal pada seluruh mahasiswa. Sebagai contoh, jika seorang mahasiswa memiliki dua mata kuliah yang dijadwalkan pada hari Selasa jam 9.00-11.00, maka state tersebut akan mendapatkan penalti sebesar 2 poin. Semakin tinggi nilai penalti, semakin banyak konflik yang dialami mahasiswa.

2. Penalti Konflik Ruang dan Waktu Berbobot

Fungsi ini mengukur konflik dari sisi penggunaan sumber daya, yaitu ruangan. Fungsi ini akan memberikan penalti ketika terdapat lebih dari satu pertemuan mata kuliah yang dijadwalkan di ruangan dan waktu yang sama. Keunikan dari fungsi ini adalah adanya pembobotan berdasarkan prioritas mata kuliah yang diambil oleh mahasiswa.

- Logika Perhitungan

Jika terjadi tabrakan jadwal di satu ruangan, penalti tidak hanya dihitung sebagai satu kejadian, tetapi diakumulasikan dari total bobot prioritas seluruh mahasiswa yang terdampak. Bobot prioritas yang digunakan adalah sebagai berikut:

- ❖ Prioritas 1: 1.75
- ❖ Prioritas 2: 1.5
- ❖ Prioritas 3: 1.25
- ❖ Lainnya: 1

Dengan demikian, tabrakan yang melibatkan mata kuliah dengan prioritas tinggi bagi banyak mahasiswa akan menghasilkan penalti yang jauh lebih besar.

3. Penalti Pelanggaran Kapasitas Ruangan

Fungsi ini memastikan bahwa jadwal yang dihasilkan memenuhi batasan fisik dan logistik dari setiap ruangan. Penalti akan diberikan jika jumlah mahasiswa yang terdaftar pada sebuah kelas melebihi kapasitas (kuota) ruangan yang dialokasikan.

- Logika Perhitungan

Penalti dihitung berdasarkan selisih antara jumlah mahasiswa dan kuota ruangan, yang kemudian dikalikan dengan jumlah SKS mata kuliah tersebut. Rumusnya adalah: $\text{Penalti} = (\text{Jumlah Mahasiswa} - \text{Kuota Ruangan}) * \text{SKS}$. Misalnya, sebuah kelas dengan 65 mahasiswa dan 2 SKS ditempatkan di ruangan berkapasitas 60. Maka penalti yang diberikan adalah $(65 - 60) * 2 = 10$ poin. Penalti SKS ditambahkan agar pelanggaran pada mata kuliah yang berdurasi lebih lama mendapatkan penalti yang lebih signifikan.

2.2. Penjelasan Implementasi Algoritma Local Search

1. Class Schedule

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Constructor	<code>data: Dict[str, Any]</code>
Deskripsi: Menginisialisasi objek jadwal dengan data yang diberikan, termasuk daftar hari dan jam yang tersedia.		
 <pre> 1 def __init__(self, data: Dict[str, Any]): 2 self.data = data 3 self.schedule = [] # List of meeting assignments 4 self.days = ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat'] 5 self.hours = list(range(7, 18)) # 7-17 (jam mulai) 6 </pre>		
<code>_get_random_sessions</code>	<code>List[int]</code>	<code>sks: int</code>

Deskripsi:

Membagi jumlah SKS menjadi beberapa sesi dengan durasi acak.

```
1 def _get_random_sessions(self, sks: int) -> List[int]:
2     """
3     Splits a total number of SKS into a List of random session durations.
4     Example: sks=4 might return [1, 2, 1] or [3, 1] or [2, 2] or [4].
5     """
6     if sks <= 0:
7         return []
8
9     sessions = []
10    remaining_sks = sks
11    while remaining_sks > 0:
12        # A session can be from 1 hour up to the remaining SKS duration
13        chunk = random.randint(1, remaining_sks)
14        sessions.append(chunk)
15        remaining_sks -= chunk
16    return sessions
```

initialize_random

List[Dict[str, Any]]

Deskripsi:

Menginisialisasi jadwal secara acak berdasarkan data kelas, ruangan, dan durasi sesi.

```
1 def initialize_random(self) -> List[Dict[str, Any]]:
2     self.schedule = []
3
4     for kelas in self.data['kelas_mata_kuliah']:
5         sks = kelas['sks']
6         sessions = self._get_random_sessions(sks)
7
8         for duration in sessions:
9             day = random.choice(self.days)
10            # Make sure end time doesn't exceed 18
11            max_start = 18 - duration
12
13            # Prevent errors if a session is too long for the day
14            if max_start < 7:
15                print(f"Warning: A {duration}-hour session for {kelas['kode']} is too long to schedule. Skipping.")
16                continue
17
18            start_hour = random.randint(7, max_start)
19            room = random.choice(self.data['ruangan'])['kode']
20
21            self.schedule.append({
22                'kode': kelas['kode'],
23                'day': day,
24                'start': start_hour,
25                'duration': duration,
26                'room': room,
27                'jumlah_mahasiswa': kelas['jumlah_mahasiswa']
28            })
29
30    return self.schedule
```

get_neighbors

List[Schedule]

Deskripsi:

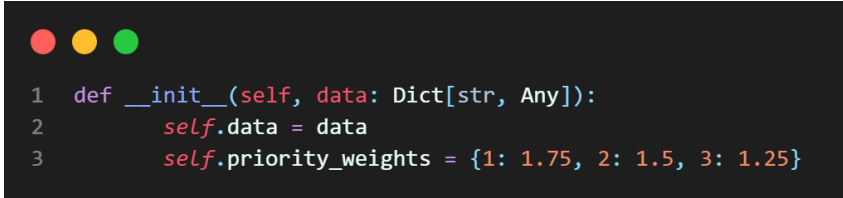
Menghasilkan daftar tetangga dari jadwal saat ini dengan dua strategi: menukar dua sesi atau memindahkan satu sesi ke slot acak.

```

1 def get_neighbors(self) -> List['Schedule']:
2     neighbors = []
3
4     # Strategy 1: Swap two meetings
5     for i in range(len(self.schedule)):
6         for j in range(i + 1, len(self.schedule)):
7             neighbor = copy.deepcopy(self)
8             # Swap times and rooms
9             neighbor.schedule[i]['day'], neighbor.schedule[j]['day'] = \
10                 neighbor.schedule[j]['day'], neighbor.schedule[i]['day']
11             neighbor.schedule[i]['start'], neighbor.schedule[j]['start'] = \
12                 neighbor.schedule[j]['start'], neighbor.schedule[i]['start']
13             neighbor.schedule[i]['room'], neighbor.schedule[j]['room'] = \
14                 neighbor.schedule[j]['room'], neighbor.schedule[i]['room']
15             neighbors.append(neighbor)
16
17     # Strategy 2: Move one meeting to random slot
18     for i in range(len(self.schedule)):
19         for _ in range(3): # Try 3 random moves per meeting
20             neighbor = copy.deepcopy(self)
21             meeting = neighbor.schedule[i]
22             meeting['day'] = random.choice(self.days)
23             max_start = 18 - meeting['duration']
24             meeting['start'] = random.randint(7, max_start)
25             meeting['room'] = random.choice(self.data['ruangan'])['kode']
26             neighbors.append(neighbor)
27
28     return neighbors

```

2. Class ObjectiveFunction

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Constructor	data: Dict[str, Any]
Deskripsi: Menginisialisasi fungsi objektif dengan data yang diberikan dan bobot prioritas untuk konflik mahasiswa.		
 <pre> 1 def __init__(self, data: Dict[str, Any]): 2 self.data = data 3 self.priority_weights = {1: 1.75, 2: 1.5, 3: 1.25} </pre>		
<code>calculate_student_conflicts</code>	float	schedule: List[Dict[str, Any]]
Deskripsi: Menghitung penalti konflik jadwal mahasiswa berdasarkan waktu yang		

tumpang tindih.

```
1 def calculate_student_conflicts(self, schedule: List[Dict[str, Any]]) -> float:
2     penalty = 0
3
4     for mahasiswa in self.data['mahasiswa']:
5         # Get all meetings for this student
6         student_meetings = [m for m in schedule if m['kode'] in mahasiswa['daftar_mk']]
7
8         # Check for conflicts between all pairs
9         for i in range(len(student_meetings)):
10             for j in range(i + 1, len(student_meetings)):
11                 m1, m2 = student_meetings[i], student_meetings[j]
12
13                 if m1['day'] == m2['day']:
14                     end1 = m1['start'] + m1['duration']
15                     end2 = m2['start'] + m2['duration']
16
17                     # Check time overlap
18                     if not (end1 <= m2['start'] or end2 <= m1['start']):
19                         overlap = min(end1, end2) - max(m1['start'], m2['start'])
20                         penalty += overlap
21
22     return penalty
```

calculate_room_
conflicts

float

schedule: List[Dict[str, Any]]

Deskripsi:

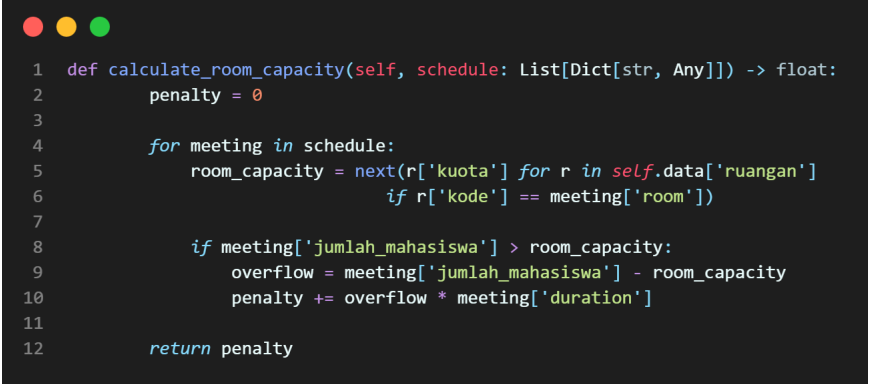
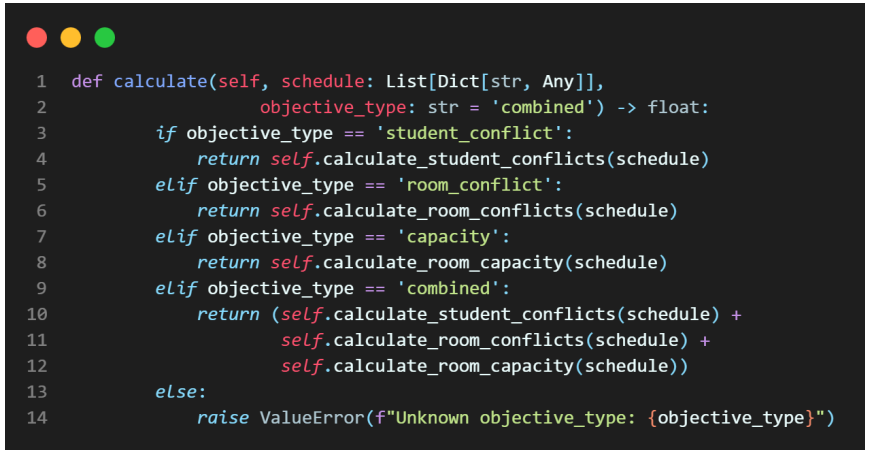
Menghitung penalti konflik ruangan berdasarkan waktu yang tumpang tindih di ruangan yang sama.

```
1 def calculate_room_conflicts(self, schedule: List[Dict[str, Any]]) -> float:
2     penalty = 0
3
4     for i in range(len(schedule)):
5         for j in range(i + 1, len(schedule)):
6             m1, m2 = schedule[i], schedule[j]
7
8             # Check if same room and overlapping time
9             if m1['room'] == m2['room'] and m1['day'] == m2['day']:
10                 end1 = m1['start'] + m1['duration']
11                 end2 = m2['start'] + m2['duration']
12
13                 if not (end1 <= m2['start'] or end2 <= m1['start']):
14                     overlap = min(end1, end2) - max(m1['start'], m2['start'])
15
16                 # Calculate weighted penalty based on student priorities
17                 for mahasiswa in self.data['mahasiswa']:
18                     if m1['kode'] in mahasiswa['daftar_mk']:
19                         idx = mahasiswa['daftar_mk'].index(m1['kode'])
20                         priority = mahasiswa['prioritas'][idx]
21                         weight = self.priority_weights.get(priority, 1.0)
22                         penalty += overlap * weight
23
24                     if m2['kode'] in mahasiswa['daftar_mk']:
25                         idx = mahasiswa['daftar_mk'].index(m2['kode'])
26                         priority = mahasiswa['prioritas'][idx]
27                         weight = self.priority_weights.get(priority, 1.0)
28                         penalty += overlap * weight
29
30     return penalty
```

calculate_room_
conflicts

float

schedule: List[Dict[str, Any]]

capacity		
Deskripsi: Menghitung penalti jika jumlah mahasiswa melebihi kapasitas ruangan.  <pre> 1 def calculate_room_capacity(self, schedule: List[Dict[str, Any]]) -> float: 2 penalty = 0 3 4 for meeting in schedule: 5 room_capacity = next(r['kuota'] for r in self.data['ruangan'] 6 if r['kode'] == meeting['room']) 7 8 if meeting['jumlah_mahasiswa'] > room_capacity: 9 overflow = meeting['jumlah_mahasiswa'] - room_capacity 10 penalty += overflow * meeting['duration'] 11 12 return penalty </pre>		
calculate	float	schedule: List[Dict[str, Any]]
Deskripsi: Menghitung nilai fungsi objektif berdasarkan tipe yang diberikan (student_conflict, room_conflict, capacity, atau combined).  <pre> 1 def calculate(self, schedule: List[Dict[str, Any]], 2 objective_type: str = 'combined') -> float: 3 if objective_type == 'student_conflict': 4 return self.calculate_student_conflicts(schedule) 5 elif objective_type == 'room_conflict': 6 return self.calculate_room_conflicts(schedule) 7 elif objective_type == 'capacity': 8 return self.calculate_room_capacity(schedule) 9 elif objective_type == 'combined': 10 return (self.calculate_student_conflicts(schedule) + 11 self.calculate_room_conflicts(schedule) + 12 self.calculate_room_capacity(schedule)) 13 else: 14 raise ValueError(f"Unknown objective_type: {objective_type}") </pre>		

3. Class HillClimbingResult

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Constructor	
Deskripsi: Menginisialisasi kelas HillClimbingResult untuk menyimpan hasil dari algoritma hill climbing, termasuk keadaan awal dan akhir, nilai fungsi		

objektif, jumlah iterasi, durasi, dan metrik lainnya.

```
1 """Class to store hill climbing results"""
2 def __init__(self):
3     self.initial_state = None
4     self.final_state = None
5     self.initial_value = None
6     self.final_value = None
7     self.iterations = 0
8     self.duration = 0.0
9     self.objective_history = []
10    self.sideways_moves = 0 # For sideways move variant
11    self.restarts = 0 # For random restart variant
12    self.iterations_per_restart = [] # For random restart variant
```

4. Class HillClimbing

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Constructor	data: Dict[str, Any], objective_type: str
Deskripsi: Menginisialisasi kelas HillClimbing dengan data dan tipe objektif yang diberikan. Fungsi ini juga menginisialisasi instance ObjectiveFunction untuk mengevaluasi keadaan		
<pre>1 def __init__(self, data: Dict[str, Any], objective_type: str = 'combined'): 2 self.data = data 3 self.objective_type = objective_type 4 self.objective_func = ObjectiveFunction(data)</pre>		
<code>_evaluate</code>	float	schedule: Schedule
Deskripsi: Menghitung nilai fungsi objektif untuk jadwal tertentu (nilai lebih rendah lebih baik)		
<pre>1 def _evaluate(self, schedule: Schedule) -> float: 2 """Evaluate objective function (Lower is better)""" 3 return self.objective_func.calculate(schedule.schedule, self.objective_type) 4</pre>		
<code>steepest_ascent</code>	HillClimbingResult	

Deskripsi:

Algoritma hill climbing yang selalu memilih tetangga terbaik hingga tidak ada perbaikan atau nilai fungsi objektif mencapai 0

```
1 def steepest_ascent(self) -> HillClimbingResult:
2     result = HillClimbingResult()
3     start_time = time.time()
4
5     current = Schedule(self.data)
6     current.initialize_random()
7     current_value = self._evaluate(current)
8
9     result.initial_state = copy.deepcopy(current.schedule)
10    result.initial_value = current_value
11    result.objective_history.append(current_value)
12
13    iteration = 0
14    while True:
15        # Stop if the objective function value is 0
16        if current_value == 0:
17            break
18
19        # Get all neighbors
20        neighbors = current.get_neighbors()
21
22        # Find the best neighbor
23        best_neighbor = None
24        best_value = current_value
25
26        for neighbor in neighbors:
27            value = self._evaluate(neighbor)
28            if value < best_value:
29                best_value = value
30                best_neighbor = neighbor
31
32        if best_neighbor is None:
33            break
34
35        # Move to best neighbor
36        current = best_neighbor
37        current_value = best_value
38        result.objective_history.append(current_value)
39        iteration += 1
```

stochastic

HillClimbingResult

Deskripsi:

Algoritma hill climbing yang memilih tetangga secara acak dari opsi yang lebih baik hingga tidak ada perbaikan atau nilai fungsi objektif mencapai 0



```

1  def stochastic(self) -> HillClimbingResult:
2      result = HillClimbingResult()
3      start_time = time.time()
4
5      current = Schedule(self.data)
6      current.initialize_random()
7      current_value = self._evaluate(current)
8
9      result.initial_state = copy.deepcopy(current.schedule)
10     result.initial_value = current_value
11     result.objective_history.append(current_value)
12
13     iteration = 0
14     while True:
15         # Stop if the objective function value is 0
16         if current_value == 0:
17             break
18
19         # Get all neighbors
20         neighbors = current.get_neighbors()
21
22         # Find all better neighbors
23         better_neighbors = []
24         for neighbor in neighbors:
25             value = self._evaluate(neighbor)
26             if value < current_value:
27                 better_neighbors.append((neighbor, value))
28
29         if not better_neighbors:
30             break
31
32         # Randomly select one better neighbor
33         selected_neighbor, selected_value = random.choice(better_neighbors)
34
35         # Move to selected neighbor
36         current = selected_neighbor
37         current_value = selected_value
38         result.objective_history.append(current_value)
39         iteration += 1
40
41     result.final_state = current.schedule
42     result.final_value = current_value
43     result.iterations = iteration
44     result.duration = time.time() - start_time
45
46     return result

```

sideways_move

HillClimbingResult

max_sideways: int

Deskripsi:

Algoritma hill climbing dengan perpindahan ke tetangga yang paling baik atau bernilai sama (sideways moves), dibatasi oleh max_sideways.


```

1  def sideways_move(self, max_sideways: int = 100) -> HillClimbingResult:
2      result = HillClimbingResult()
3      start_time = time.time()
4
5      current = Schedule(self.data)
6      current.initialize_random()
7      current_value = self._evaluate(current)
8
9      result.initial_state = copy.deepcopy(current.schedule)
10     result.initial_value = current_value
11     result.objective_history.append(current_value)
12
13     iteration = 0
14     consecutive_sideways = 0
15
16     while True:
17         if current_value == 0:
18             break
19
20         # Check sideways limit before processing
21         if consecutive_sideways >= max_sideways:
22             break
23
24         neighbors = current.get_neighbors()
25
26         best_neighbors = []
27         best_value = float('inf')
28
29         # Find all neighbors with best value
30         for neighbor in neighbors:
31             value = self._evaluate(neighbor)
32             if value < best_value:
33                 best_value = value
34                 best_neighbors = [neighbor]
35             elif value == best_value:
36                 best_neighbors.append(neighbor)
37
38         # If no improvement or sideways move possible
39         if best_value > current_value:
40             break
41
42         # Randomly select from best neighbors
43         best_neighbor = random.choice(best_neighbors)
44
45         # Track if this is a sideways move
46         if best_value == current_value:
47             consecutive_sideways += 1
48             result.sideways_moves += 1
49         else:
50             consecutive_sideways = 0
51
52         current = best_neighbor
53         current_value = best_value
54         result.objective_history.append(current_value)
55         iteration += 1
56
57     result.final_state = current.schedule
58     result.final_value = current_value
59     result.iterations = iteration
60     result.duration = time.time() - start_time
61
62     return result

```

random_restart	HillClimbingResult	max_restarts: int (default: 10), max_iterations_per_restart: int
----------------	--------------------	---

Deskripsi:

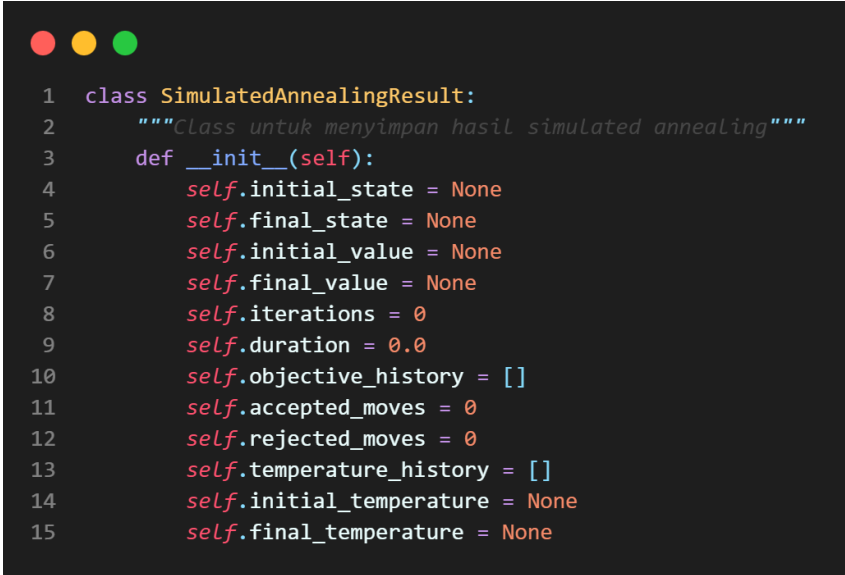
Algoritma hill climbing dengan beberapa restart acak hingga solusi optimal ditemukan atau batas restart tercapai

```

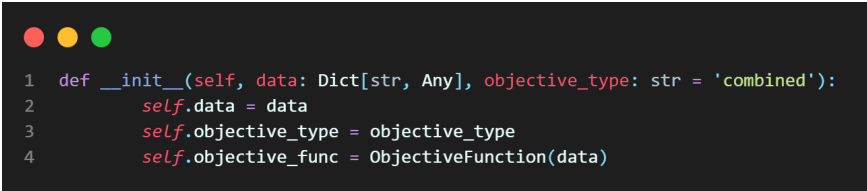
1  def random_restart(self, max_restarts: int = 10, max_iterations_per_restart: int = 100) -> HillClimbingResult:
2      result = HillClimbingResult()
3      start_time = time.time()
4
5      best_overall_state = None
6      best_overall_value = float('inf')
7      all_objective_history = []
8
9      first_schedule = Schedule(self.data)
10     first_schedule.initialize_random()
11     result.initial_state = copy.deepcopy(first_schedule.schedule)
12     result.initial_value = self._evaluate(first_schedule)
13
14     actual_restarts = 0
15     for restart in range(max_restarts):
16         actual_restarts = restart + 1
17
18         if restart == 0:
19             current = first_schedule
20         else:
21             current = Schedule(self.data)
22             current.initialize_random()
23
24         current_value = self._evaluate(current)
25         iteration = 0
26
27         while iteration < max_iterations_per_restart:
28             if current_value == 0:
29                 best_overall_state = current.schedule
30                 best_overall_value = current_value
31                 break
32
33             neighbors = current.get_neighbors()
34
35             best_neighbor = None
36             best_value = current_value
37
38             for neighbor in neighbors:
39                 value = self._evaluate(neighbor)
40                 if value < best_value:
41                     best_value = value
42                     best_neighbor = neighbor
43
44             if best_neighbor is None:
45                 break
46
47             current = best_neighbor
48             current_value = best_value
49             all_objective_history.append(current_value)
50             iteration += 1
51
52         result.iterations_per_restart.append(iteration)
53
54         if current_value < best_overall_value:
55             best_overall_value = current_value
56             best_overall_state = current.schedule
57
58         if best_overall_value == 0:
59             break
60
61     result.final_state = best_overall_state
62     result.final_value = best_overall_value
63     result.restarts = actual_restarts
64     result.iterations = sum(result.iterations_per_restart)
65     result.objective_history = all_objective_history
66     result.duration = time.time() - start_time
67
68     return result

```

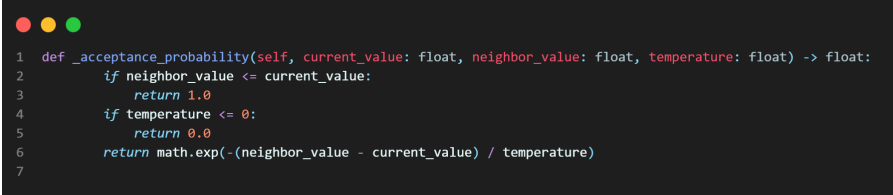
5. Class SimulatedAnnealingResult

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Constructor	
Deskripsi: Menginisialisasi objek untuk menyimpan hasil dari algoritma simulated annealing, termasuk keadaan awal dan akhir, nilai fungsi objektif, jumlah iterasi, durasi, riwayat nilai objektif, jumlah perpindahan yang diterima dan ditolak, serta riwayat temperatur.		
 <pre> 1 class SimulatedAnnealingResult: 2 """Class untuk menyimpan hasil simulated annealing""" 3 def __init__(self): 4 self.initial_state = None 5 self.final_state = None 6 self.initial_value = None 7 self.final_value = None 8 self.iterations = 0 9 self.duration = 0.0 10 self.objective_history = [] 11 self.accepted_moves = 0 12 self.rejected_moves = 0 13 self.temperature_history = [] 14 self.initial_temperature = None 15 self.final_temperature = None </pre>		

6. Class SimulatedAnnealing

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Constructor	data: Dict[str, Any], objective_type: str
Deskripsi: Menginisialisasi algoritma simulated annealing dengan data dan tipe fungsi objektif yang diberikan. Fungsi ini juga menginisialisasi instance ObjectiveFunction untuk mengevaluasi keadaan.		
 <pre> 1 def __init__(self, data: Dict[str, Any], objective_type: str = 'combined'): 2 self.data = data 3 self.objective_type = objective_type 4 self.objective_func = ObjectiveFunction(data) </pre>		

_evaluate	float	schedule: Schedule
Deskripsi: Menghitung nilai fungsi objektif untuk jadwal tertentu (nilai lebih rendah lebih baik).  <pre> 1 def _evaluate(self, schedule: Schedule) -> float: 2 return self.objective_func.calculate(schedule.schedule, self.objective_type) </pre>		
_cooling_schedule_linear	float	initial_temp: float, final_temp: float, current_iteration: int, max_iterations: int
Deskripsi: Menghitung temperatur baru menggunakan jadwal penurunan linear.  <pre> 1 def _cooling_schedule_linear(self, initial_temp: float, final_temp: float, current_iteration: int, max_iterations: int) -> float: 2 if current_iteration >= max_iterations: 3 return final_temp 4 return initial_temp - (initial_temp - final_temp) * (current_iteration / max_iterations) </pre>		
_cooling_schedule_exponential	float	initial_temp: float, alpha: float, current_iteration: int
Deskripsi: Menghitung temperatur baru menggunakan jadwal penurunan eksponensial.  <pre> 1 def _cooling_schedule_exponential(self, initial_temp: float, alpha: float, current_iteration: int) -> float: 2 return initial_temp * (alpha ** current_iteration) </pre>		
_cooling_schedule_logarithmic	float	initial_temp: float, current_iteration: int
Deskripsi: Menghitung temperatur baru menggunakan jadwal penurunan logaritmik.  <pre> 1 def _cooling_schedule_logarithmic(self, initial_temp: float, current_iteration: int) -> float: 2 if current_iteration == 0: 3 return initial_temp </pre>		

_acceptance_probability	float	current_value: float, neighbor_value: float, temperature: float
Deskripsi: Menghitung probabilitas untuk menerima tetangga berdasarkan perbedaan nilai fungsi objektif dan temperatur.  <pre> 1 def _acceptance_probability(self, current_value: float, neighbor_value: float, temperature: float) -> float: 2 if neighbor_value <= current_value: 3 return 1.0 4 if temperature <= 0: 5 return 0.0 6 return math.exp(-(neighbor_value - current_value) / temperature) 7 </pre>		
run	SimulatedAnnealing Result	initial_temperature: float, final_temperature: float, max_iterations: int, cooling_schedule: str, alpha: float
Deskripsi: Menjalankan algoritma simulated annealing dengan berbagai jadwal penurunan temperatur (cooling schedule). Proses berhenti ketika temperatur mencapai batas akhir atau iterasi maksimum tercapai.		

```

1 def run(self, initial_temperature: float = 1000.0, final_temperature: float = 0.1, max_iterations: int = 1000, cooling_schedule: str = 'exponential', alpha: float = 0.95) -> SimulatedAnnealingResult:
2     result = SimulatedAnnealingResult()
3     start_time = time.time()
4
5     # Initialize
6     current = Schedule(self.data)
7     current.initialize_random()
8     current_value = self._evaluate(current)
9
10    best = copy.deepcopy(current)
11    best_value = current_value
12
13    result.initial_state = copy.deepcopy(current.schedule)
14    result.initial_value = current_value
15    result.initial_temperature = initial_temperature
16    result.final_temperature = final_temperature
17    result.objective_history.append(current_value)
18
19    temperature = initial_temperature
20    iteration = 0
21
22    while iteration < max_iterations and temperature > final_temperature:
23        # Generate random neighbor
24        neighbors = current.get_neighbors()
25        if not neighbors:
26            break
27
28        neighbor = random.choice(neighbors)
29        neighbor_value = self._evaluate(neighbor)
30
31        # Hitung probability move
32        accept_prob = self._acceptance_probability(current_value, neighbor_value, temperature)
33
34        # terima/tolak move
35        if random.random() < accept_prob:
36            current = neighbor
37            current_value = neighbor_value
38            result.accepted_moves += 1
39
40            if current_value < best_value:
41                best = copy.deepcopy(current)
42                best_value = current_value
43        else:
44            result.rejected_moves += 1
45
46        # simpan ke history
47        result.objective_history.append(current_value)
48        result.temperature_history.append(temperature)
49
50        # update temperature berdasarkan jenis penurunan temp
51        if cooling_schedule == 'linear':
52            temperature = self._cooling_schedule_linear(initial_temperature, final_temperature, iteration + 1, max_iterations)
53        elif cooling_schedule == 'exponential':
54            temperature = self._cooling_schedule_exponential(initial_temperature, alpha, iteration + 1)
55        elif cooling_schedule == 'logarithmic':
56            temperature = self._cooling_schedule_logarithmic(initial_temperature, iteration + 1)
57        else:
58            raise ValueError(f"Unknown cooling schedule: {cooling_schedule}")
59
60        iteration += 1
61
62    result.final_state = best.schedule
63    result.final_value = best_value
64    result.iterations = iteration
65    result.duration = time.time() - start_time
66
67    return result

```

run_linear_cooling

SimulatedAnnealing
Result

initial_temperature: float,
final_temperature: float,
max_iterations: int

Deskripsi:

Menjalankan algoritma simulated annealing dengan jadwal penurunan temperatur secara linear.

```

1 def run_linear_cooling(self, initial_temperature: float = 1000.0, final_temperature: float = 0.1, max_iterations: int = 1000) -> SimulatedAnnealingResult:
2     return self.run(initial_temperature, final_temperature, max_iterations, 'linear')
3

```

run_exponential_cooling

SimulatedAnnealing
Result

initial_temperature: float,
alpha: float,
max_iterations: int

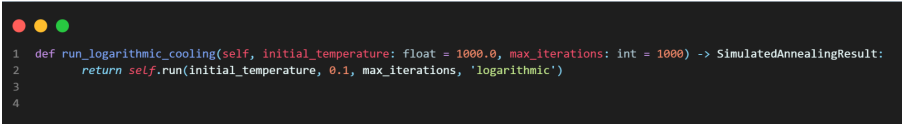
Deskripsi:

Menjalankan algoritma simulated annealing dengan jadwal penurunan temperatur secara eksponensial.

```

1 def run_exponential_cooling(self, initial_temperature: float = 1000.0, alpha: float = 0.95, max_iterations: int = 1000) -> SimulatedAnnealingResult:
2     return self.run(initial_temperature, 0.1, max_iterations, 'exponential', alpha)
3

```

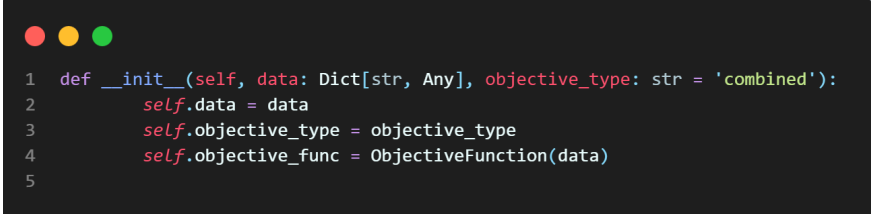
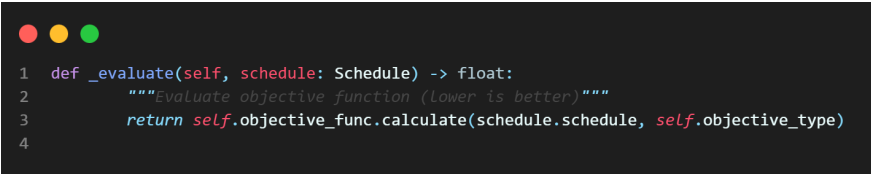
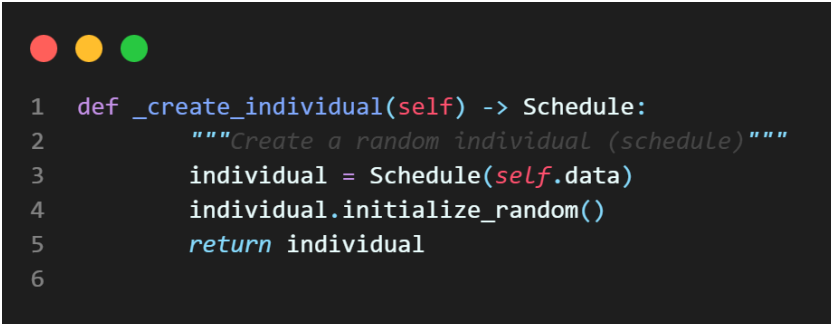
run_logarithmic_cooling	SimulatedAnnealing Result	initial_temperature: float, max_iterations: int
Deskripsi: Menjalankan algoritma simulated annealing dengan jadwal penurunan temperatur secara logaritmik.  <pre> 1 def run_logarithmic_cooling(self, initial_temperature: float = 1000.0, max_iterations: int = 1000) -> SimulatedAnnealingResult: 2 return self.run(initial_temperature, 0.1, max_iterations, 'logarithmic') 3 4 </pre>		

7. Class GeneticResult

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Constructor	
Deskripsi: Menginisialisasi objek untuk menyimpan hasil dari algoritma genetika, termasuk keadaan awal dan akhir, nilai fungsi objektif, jumlah iterasi, durasi, riwayat fitness terbaik, dan rata-rata fitness.  <pre> 1 def __init__(self): 2 self.initial_state = None 3 self.final_state = None 4 self.initial_value = None 5 self.final_value = None 6 self.iterations = 0 7 self.duration = 0.0 8 self.objective_history = [] 9 self.population_size = 0 10 self.best_fitness_history = [] 11 self.average_fitness_history = [] </pre>		

8. Class GeneticAlgorithm

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Constructor	data: Dict[str, Any],

		objective_type: str
Deskripsi: Menginisialisasi algoritma genetika dengan data dan tipe fungsi objektif yang diberikan.		
 <pre> 1 def __init__(self, data: Dict[str, Any], objective_type: str = 'combined'): 2 self.data = data 3 self.objective_type = objective_type 4 self.objective_func = ObjectiveFunction(data) 5 </pre>		
_evaluate	float	schedule: Schedule
Deskripsi: Menghitung nilai fungsi objektif untuk jadwal tertentu (nilai lebih rendah lebih baik).		
 <pre> 1 def _evaluate(self, schedule: Schedule) -> float: 2 """Evaluate objective function (Lower is better)""" 3 return self.objective_func.calculate(schedule.schedule, self.objective_type) 4 </pre>		
_create_individual	Schedule	
Deskripsi: Membuat individu baru secara acak (jadwal acak).		
 <pre> 1 def _create_individual(self) -> Schedule: 2 """Create a random individual (schedule)""" 3 individual = Schedule(self.data) 4 individual.initialize_random() 5 return individual 6 </pre>		
_initialize_population	List[Schedule]	population_size: int
Deskripsi:		

Menginisialisasi populasi awal dengan sejumlah individu acak.

```
1 def _initialize_population(self, population_size: int) -> List[Schedule]:
2     """Initialize population with random individuals"""
3     population = []
4     for _ in range(population_size):
5         individual = self._create_individual()
6         population.append(individual)
7     return population
```

_selection_roulette

Schedule

population: List[Schedule]

Deskripsi:

Melakukan seleksi individu menggunakan metode roulette wheel selection berdasarkan nilai fitness.

```
1 def _selection_roulette(self, population: List[Schedule]) -> Schedule:
2     """Roulette wheel selection - probability based on fitness"""
3     # Convert to fitness values (higher is better)
4     fitness_values = []
5     max_penalty = max(self._evaluate(ind) for ind in population) + 1
6
7     for individual in population:
8         penalty = self._evaluate(individual)
9         fitness = max_penalty - penalty # Convert penalty to fitness
10        fitness_values.append(max(fitness, 0.1)) # Ensure positive fitness
11
12        total_fitness = sum(fitness_values)
13        if total_fitness == 0:
14            return copy.deepcopy(random.choice(population))
15
16        # Roulette wheel selection
17        spin = random.uniform(0, total_fitness)
18        current = 0
19
20        for i, fitness in enumerate(fitness_values):
21            current += fitness
22            if current >= spin:
23                return copy.deepcopy(population[i])
24
25        return copy.deepcopy(population[-1])
```

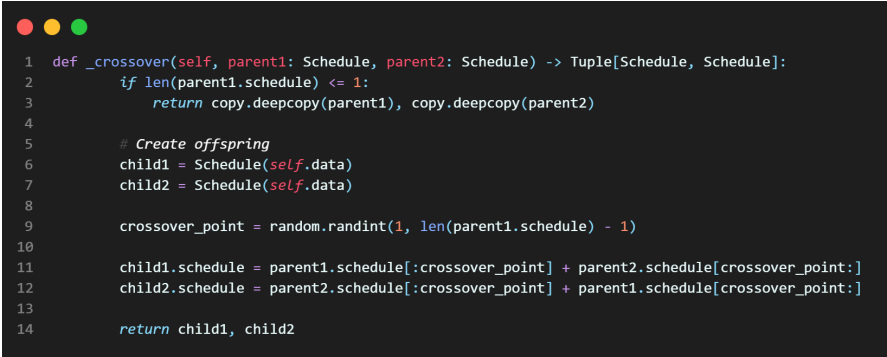
_crossover

Tuple[Schedule,
Schedule]

parent1: Schedule,
parent2: Schedule

Deskripsi:

Melakukan crossover antara dua individu untuk menghasilkan dua individu baru.



```

1  def _crossover(self, parent1: Schedule, parent2: Schedule) -> Tuple[Schedule, Schedule]:
2      if len(parent1.schedule) <= 1:
3          return copy.deepcopy(parent1), copy.deepcopy(parent2)
4
5      # Create offspring
6      child1 = Schedule(self.data)
7      child2 = Schedule(self.data)
8
9      crossover_point = random.randint(1, len(parent1.schedule) - 1)
10
11     child1.schedule = parent1.schedule[:crossover_point] + parent2.schedule[crossover_point:]
12     child2.schedule = parent2.schedule[:crossover_point] + parent1.schedule[crossover_point:]
13
14     return child1, child2

```

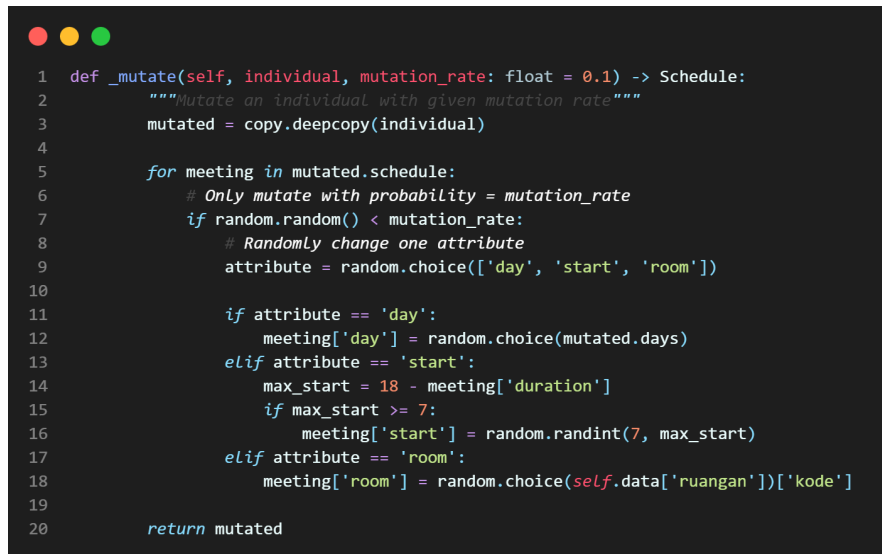
_mutate

Schedule

individual: Schedule,
mutation_rate: float

Deskripsi:

Melakukan mutasi pada individu dengan probabilitas tertentu.



```

1  def _mutate(self, individual, mutation_rate: float = 0.1) -> Schedule:
2      """Mutate an individual with given mutation rate"""
3      mutated = copy.deepcopy(individual)
4
5      for meeting in mutated.schedule:
6          # Only mutate with probability = mutation_rate
7          if random.random() < mutation_rate:
8              # Randomly change one attribute
9              attribute = random.choice(['day', 'start', 'room'])
10
11             if attribute == 'day':
12                 meeting['day'] = random.choice(mutated.days)
13             elif attribute == 'start':
14                 max_start = 18 - meeting['duration']
15                 if max_start >= 7:
16                     meeting['start'] = random.randint(7, max_start)
17             elif attribute == 'room':
18                 meeting['room'] = random.choice(self.data['ruangan'])['kode']
19
20     return mutated

```

run

GeneticResult

population_size: int,
max_iterations: int,
mutation_rate: float

Deskripsi:

Menjalankan algoritma genetika dengan parameter yang ditentukan. Proses melibatkan inisialisasi populasi, seleksi, crossover, mutasi, dan evaluasi hingga generasi maksimum tercapai.

```

1  def run(self, population_size: int = 50, max_iterations: int = 100,
2      mutation_rate: float = 0.1) -> GeneticResult:
3      """
4      Run genetic algorithm with specified parameters
5
6      Args:
7          population_size: Size of the population
8          max_iterations: Maximum number of generations
9          mutation_rate: Probability of mutation for each gene (default: 0.1)
10     """
11     result = GeneticResult()
12     start_time = time.time()
13
14     # Store parameters
15     result.population_size = population_size
16
17     # Initialize population
18     population = self._initialize_population(population_size)
19
20     # Evaluate initial population
21     initial_best = min(population, key=lambda x: self._evaluate(x))
22     result.initial_state = copy.deepcopy(initial_best.schedule)
23     result.initial_value = self._evaluate(initial_best)
24
25     # Evolution Loop
26     for generation in range(max_iterations):
27         # Evaluate population
28         fitness_values = [self._evaluate(ind) for ind in population]
29         best_fitness = min(fitness_values)
30         avg_fitness = sum(fitness_values) / len(fitness_values)
31
32         # Store statistics
33         result.best_fitness_history.append(best_fitness)
34         result.average_fitness_history.append(avg_fitness)
35         result.objective_history.append(best_fitness)
36
37         # Create new generation
38         offspring = []
39
40         while len(offspring) < population_size:
41             # Selection
42             parent1 = self._selection_roulette(population)
43             parent2 = self._selection_roulette(population)
44
45             # Crossover
46             child1, child2 = self._crossover(parent1, parent2)
47
48             # Mutation
49             child1 = self._mutate(child1, mutation_rate)
50             child2 = self._mutate(child2, mutation_rate)
51
52             offspring.extend([child1, child2])
53
54         # Limit offspring to population size
55         offspring = offspring[:population_size]
56
57         # Replacement
58         population = offspring
59
60     # Final evaluation
61     final_best = min(population, key=lambda x: self._evaluate(x))
62     result.final_state = final_best.schedule
63     result.final_value = self._evaluate(final_best)
64     result.iterations = max_iterations
65     result.duration = time.time() - start_time
66
67     return result
68

```

9. Parser

Nama Fungsi	Tipe	Parameter
load_json	Dict[str, Any]	path: str
Deskripsi: Membaca file JSON dari path yang diberikan dan mengembalikan data dalam bentuk dictionary. <pre>1 def load_json(path: str) -> Dict[str, Any]: 2 with open(path, "r", encoding="utf-8") as f: 3 return json.load(f)</pre>		
_int_checker	int	v: Any, default: int
Deskripsi: Mengonversi nilai ke integer. Jika gagal, mengembalikan nilai default. <pre>1 def _int_checker(v: Any, default: int = 0) -> int: 2 try: 3 return int(v) 4 except Exception: 5 return default</pre>		
_normalize_kelas	Dict[str, Any]	item: Dict[str, Any]
Deskripsi: Menormalisasi data kelas menjadi dictionary dengan kunci kode, jumlah_mahasiswa, dan sks. <pre>1 def _normalize_kelas(item: Dict[str, Any]) -> Dict[str, Any]: 2 kode = str(item.get("kode", "")).strip() 3 jumlah_mahasiswa = _int_checker(item.get("jumlah_mahasiswa"), 0) 4 sks = _int_checker(item.get("sks"), 0) 5 return {"kode": kode, "jumlah_mahasiswa": jumlah_mahasiswa, "sks": sks}</pre>		

_normalize_ruangan	Dict[str, Any]	item: Dict[str, Any]
Deskripsi: Menormalisasi data ruangan menjadi dictionary dengan kunci kode dan kuota. <pre> 1 def _normalize_ruangan(item: Dict[str, Any]) -> Dict[str, Any]: 2 kode = item.get("kode") 3 if kode is None or str(kode).strip() == "": 4 kode = str(item.get("name") or item.get("nama") or "").strip() 5 kode = str(kode) 6 kuota = _int_checker(item.get("kuota"), 0) 7 return {"kode": kode, "kuota": kuota} </pre>		
_normalize_mahasiswa	Dict[str, Any]	item: Dict[str, Any]
Deskripsi: Menormalisasi data mahasiswa menjadi dictionary dengan kunci nim, daftar_mk, dan prioritas. <pre> 1 def _normalize_mahasiswa(item: Dict[str, Any]) -> Dict[str, Any]: 2 nim = str(item.get("nim", "")).strip() 3 daftar_mk = item.get("daftar_mk") or item.get("daftar_matkul") or [] 4 daftar_mk = [str(x) for x in daftar_mk] 5 prioritas = item.get("prioritas") or [] 6 seen = set() 7 cleaned: List[int] = [] 8 for p in prioritas: 9 try: 10 pi = int(p) 11 except Exception: 12 continue 13 if pi in seen: 14 continue 15 seen.add(pi) 16 cleaned.append(pi) 17 return {"nim": nim, "daftar_mk": daftar_mk, "prioritas": cleaned} </pre>		
parse_data	Dict[str, Any]	data: Dict[str, Any]
Deskripsi: Memproses data mentah menjadi data yang sudah dinormalisasi, termasuk data kelas, ruangan, dan mahasiswa.		



```

1 def parse_data(data: Dict[str, Any]) -> Dict[str, Any]:
2     output: Dict[str, Any] = {}
3
4     kmk = data.get("kelas_mata_kuliah") or data.get("kelas") or []
5     output["kelas_mata_kuliah"] = [_normalize_kelas(x) for x in kmk]
6
7     ruangan = data.get("ruangan") or data.get("ruang") or []
8     output["ruangan"] = [_normalize_ruangan(x) for x in ruangan]
9
10    mahasiswa = data.get("mahasiswa") or data.get("mahasiswas") or []
11    output["mahasiswa"] = [_normalize_mahasiswa(x) for x in mahasiswa]
12
13    return output

```

load_and_parse

Dict[str, Any]

path: str

Deskripsi:

Membaca file JSON dari path yang diberikan, lalu memprosesnya menjadi data yang sudah dinormalisasi.



```

1 def load_and_parse(path: str) -> Dict[str, Any]:
2     data = load_json(path)
3     return parse_data(data)

```

10. Output Formatter

Nama Fungsi	Tipe	Parameter
print_summary	None	data: dict
Deskripsi: Menampilkan ringkasan hasil parsing data, termasuk daftar kelas, ruangan, dan mahasiswa.		

```

1 def print_summary(data: dict) -> None:
2     print("Hasil parsing input:\n")
3
4     kelas_list = data['kelas_mata_kuliah']
5     print(f"Kelas mata kuliah ({len(kelas_list)}):")
6     for k in kelas_list:
7         print(f" - {k['kode']}: jumlah_mahasiswa={k['jumlah_mahasiswa']}, sks={k['sks']}")
8
9     ruangan_list = data['ruangan']
10    print(f"\nRuangan ({len(ruangan_list)}):")
11    for r in ruangan_list:
12        print(f" - {r['kode']}: kuota={r['kuota']}")
13
14    mahasiswa_list = data['mahasiswa']
15    print(f"\nMahasiswa ({len(mahasiswa_list)}):")
16    for m in mahasiswa_list:
17        print(f" - {m['nim']}: daftar_mk={m['daftar_mk']}, prioritas={m['prioritas']}")
18

```

print_schedule

None

schedule: List[Dict[str, Any]],
title: str

Deskripsi:

Menginisialisasi kelas HillClimbingResult untuk menyimpan hasil dari algoritma hill climbing, termasuk keadaan awal dan akhir, nilai fungsi objektif, jumlah iterasi, durasi, dan metrik lainnya.

```

1 def print_schedule(schedule: List[Dict[str, Any]], title: str):
2     print(f"\n{'='*60}")
3     print(f"(title)")
4     print(f"{'='*60}")
5
6     rooms = {}
7     for meeting in schedule:
8         room = meeting['room']
9         if room not in rooms:
10            rooms[room] = []
11            rooms[room].append(meeting)
12
13    for room, meetings in sorted(rooms.items()):
14        print(f"\nRuangan: {room}")
15        print(f"{'Jam':<5} {'Senin':<15} {'Selasa':<15} {'Rabu':<15} {'Kamis':<15} {'Jumat':<15}")
16        print("-" * 85)
17        days = ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat']
18        for hour in range(7, 18):
19            row = [f"{'hour':<5}"]
20            for day in days:
21                meeting_code = ""
22                for m in meetings:
23                    if m['day'] == day and m['start'] <= hour < m['start'] + m['duration']:
24                        meeting_code = m['kode']
25                        break
26                row.append(f"{'meeting_code':<15}")
27            print("".join(row))

```

print_hill_climbing_result

None

result: HillClimbingResult,
algorithm_name: str,
run_num: int

Deskripsi:

Menampilkan hasil eksperimen algoritma hill climbing, termasuk nilai fungsi objektif awal dan akhir, jumlah iterasi, durasi, serta informasi tambahan seperti jumlah sideways moves dan restarts.

```
1 def print_hill_climbing_result(result: HillClimbingResult, algorithm_name: str, run_num: int):
2     print(f"\n{'='*60}")
3     print(f"{algorithm_name} (Run {run_num}) - HASIL EKSPERIMEN")
4     print(f"{'='*60}")
5     print(f"Nilai Objective Function Awal: {result.initial_value:.2f}")
6     print(f"Nilai Objective Function Akhir: {result.final_value:.2f}")
7     print(f"Improvement: {result.initial_value - result.final_value:.2f}")
8     print(f"Jumlah Iterasi: {result.iterations}")
9     print(f"Durasi: {result.duration:.4f} detik")
10
11     if hasattr(result, 'sideways_moves') and result.sideways_moves > 0:
12         print(f"Sideways Moves: {result.sideways_moves}")
13
14     if hasattr(result, 'restarts') and result.restarts > 0:
15         print(f"Jumlah Restart: {result.restarts}")
16         print(f"Iterasi per Restart: {result.iterations_per_restart}")
```

print_simulated_annealing_result

None

result:
SimulatedAnnealingResult,
algorithm_name: str,
run_num: int

Deskripsi:

Menampilkan hasil eksperimen algoritma simulated annealing, termasuk nilai fungsi objektif awal dan akhir, jumlah iterasi, durasi, temperatur awal dan akhir, serta jumlah perpindahan yang diterima dan ditolak.

```
1 def print_simulated_annealing_result(result: SimulatedAnnealingResult, algorithm_name: str, run_num: int):
2     print(f"\n{'='*60}")
3     print(f"{algorithm_name} (Run {run_num}) - HASIL EKSPERIMEN")
4     print(f"{'='*60}")
5     print(f"Nilai Objective Function Awal: {result.initial_value:.2f}")
6     print(f"Nilai Objective Function Akhir: {result.final_value:.2f}")
7     print(f"Improvement: {result.initial_value - result.final_value:.2f}")
8     print(f"Jumlah Iterasi: {result.iterations}")
9     print(f"Durasi: {result.duration:.4f} detik")
10    print(f"Temperature Awal: {result.initial_temperature:.2f}")
11    print(f"Temperature Akhir: {result.final_temperature:.2f}")
12    print(f"Accepted Moves: {result.accepted_moves}")
13    print(f"Rejected Moves: {result.rejected_moves}")
14    if result.accepted_moves + result.rejected_moves > 0:
15        acceptance_rate = result.accepted_moves / (result.accepted_moves + result.rejected_moves) * 100
16        print(f"Acceptance Rate: {acceptance_rate:.2f}%")
```

print_genetic_result

None

result: GeneticResult,
run_num: int

Deskripsi:

Menampilkan hasil eksperimen algoritma genetika, termasuk nilai fungsi objektif awal dan akhir, jumlah generasi, durasi, ukuran populasi, serta

riwayat fitness terbaik dan rata-rata fitness.

```
1 def print_genetic_result(result: GeneticResult, run_num: int):
2     print(f"\n{'='*60}")
3     print(f"Genetic Algorithm (Run {run_num}) - HASIL EKSPERIMEN")
4     print(f"{'='*60}")
5     print(f"Nilai Objective Function Awal: {result.initial_value:.2f}")
6     print(f"Nilai Objective Function Akhir: {result.final_value:.2f}")
7     print(f"Improvement: {result.initial_value - result.final_value:.2f}")
8     print(f"Jumlah Iterasi: {result.iterations}")
9     print(f"Durasi: {result.duration:.4f} detik")
10    print(f"Ukuran Populasi: {result.population_size}")
11    print(f"Final Average Fitness: {result.average_fitness_history[-1]:.2f}")
```

11. Plot

Nama Fungsi	Tipe	Parameter
plot_objective_history	None	result, algorithm_name: str, run_num: int
Deskripsi: Membuat plot nilai fungsi objektif terhadap iterasi untuk algoritma tertentu.		

```

1  def plot_objective_history(result, algorithm_name: str, run_num: int):
2      if not hasattr(result, 'objective_history') or not result.objective_history:
3          print(f"No objective history available for {algorithm_name}")
4          return
5
6      plt.figure(figsize=(10, 6))
7
8      iterations = list(range(len(result.objective_history)))
9      plt.plot(iterations, result.objective_history, 'b-', linewidth=2, label='Objective Value')
10
11     # Add markers for initial and final values
12     plt.plot(0, result.initial_value, 'go', markersize=10, label=f'Initial: {result.initial_value:.2f}')
13     plt.plot(len(result.objective_history)-1, result.final_value, 'ro', markersize=10,
14             label=f'Final: {result.final_value:.2f}')
15
16     plt.xlabel('Iteration', fontsize=12)
17     plt.ylabel('Objective Function Value', fontsize=12)
18     plt.title(f'{algorithm_name} - Objective Function vs Iterations (Run {run_num})', fontsize=14)
19     plt.legend(fontsize=10)
20     plt.grid(True, alpha=0.3)
21
22     # Add statistics box
23     improvement = result.initial_value - result.final_value
24     stats_text = f'Iterations: {result.iterations}\n'
25     stats_text += f'Improvement: {improvement:.2f}\n'
26     stats_text += f'Duration: {result.duration:.3f}s'
27
28     plt.text(0.02, 0.98, stats_text, transform=plt.gca().transAxes,
29            fontsize=10, verticalalignment='top',
30            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
31
32     plt.tight_layout()
33
34     # Save to plots folder with timestamp
35     timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
36     current_dir = os.path.dirname(os.path.abspath(__file__))
37     project_root = os.path.dirname(current_dir)
38     plots_dir = os.path.join(project_root, 'plots')
39
40     if not os.path.exists(plots_dir):
41         os.makedirs(plots_dir)
42
43     safe_algo_name = algorithm_name.replace(' ', '_').replace('(', '').replace(')', '')
44     filename = f"{timestamp}_{safe_algo_name}_run{run_num}.png"
45     save_path = os.path.join(plots_dir, filename)
46
47     plt.savefig(save_path, dpi=300, bbox_inches='tight')
48
49     plt.close()

```

plot_genetic_populati
on_fitness

None

result,
run_num: int

Deskripsi:

Membuat plot nilai fitness terbaik dan rata-rata fitness populasi terhadap generasi untuk algoritma genetika.

```

1 def plot_genetic_population_fitness(result, run_num: int):
2     if not hasattr(result, 'best_fitness_history') or not result.best_fitness_history:
3         print("No fitness history available for Genetic Algorithm")
4         return
5
6     if not hasattr(result, 'average_fitness_history') or not result.average_fitness_history:
7         print("No average fitness history available for Genetic Algorithm")
8         return
9
10    plt.figure(figsize=(12, 7))
11
12    generations = list(range(len(result.best_fitness_history)))
13
14    plt.plot(generations, result.best_fitness_history, 'b-', linewidth=2.5,
15             label='Best Fitness', marker='o', markersize=4,
16             markevery=max(1, len(generations)//20))
17
18    plt.plot(generations, result.average_fitness_history, 'r--', linewidth=2.5,
19             label='Average Fitness', marker='s', markersize=4,
20             markevery=max(1, len(generations)//20))
21
22    plt.plot(0, result.best_fitness_history[0], 'go', markersize=12,
23             label=f'Initial Best: {result.best_fitness_history[0]:.2f}', zorder=5)
24    plt.plot(len(generations)-1, result.best_fitness_history[-1], 'mo', markersize=12,
25             label=f'Final Best: {result.best_fitness_history[-1]:.2f}', zorder=5)
26
27    plt.xlabel('Generasi', fontsize=13)
28    plt.ylabel('Objective Function Value', fontsize=13)
29    plt.title(f'Genetic Algorithm - Best & Average Fitness vs Generasi (Run {run_num})',
30             fontsize=14, fontweight='bold')
31    plt.legend(fontsize=10, loc='best')
32    plt.grid(True, alpha=0.3)
33
34    improvement = result.best_fitness_history[0] - result.best_fitness_history[-1]
35    avg_improvement = result.average_fitness_history[0] - result.average_fitness_history[-1]
36
37    stats_text = f'Population Size: {result.population_size}\n'
38    stats_text += f'Generations: {result.iterations}\n'
39    stats_text += f'Best Improvement: {improvement:.2f}\n'
40    stats_text += f'Avg Improvement: {avg_improvement:.2f}\n'
41    stats_text += f'Final Best: {result.best_fitness_history[-1]:.2f}\n'
42    stats_text += f'Final Average: {result.average_fitness_history[-1]:.2f}\n'
43    stats_text += f'Duration: {result.duration:.3f}s'
44
45    plt.text(0.02, 0.98, stats_text, transform=plt.gca().transAxes,
46            fontsize=9, verticalalignment='top',
47            bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.7))
48
49    plt.tight_layout()
50
51    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
52    current_dir = os.path.dirname(os.path.abspath(__file__))
53    project_root = os.path.dirname(current_dir)
54    plots_dir = os.path.join(project_root, 'plots')
55
56    if not os.path.exists(plots_dir):
57        os.makedirs(plots_dir)
58
59    filename = f"{timestamp}_Genetic_Algorithm_run{run_num}.png"
60    save_path = os.path.join(plots_dir, filename)
61
62    plt.savefig(save_path, dpi=300, bbox_inches='tight')
63
64    plt.close()

```

plot_simulated_annealing_eT

None

result,
run_num: int

Deskripsi:

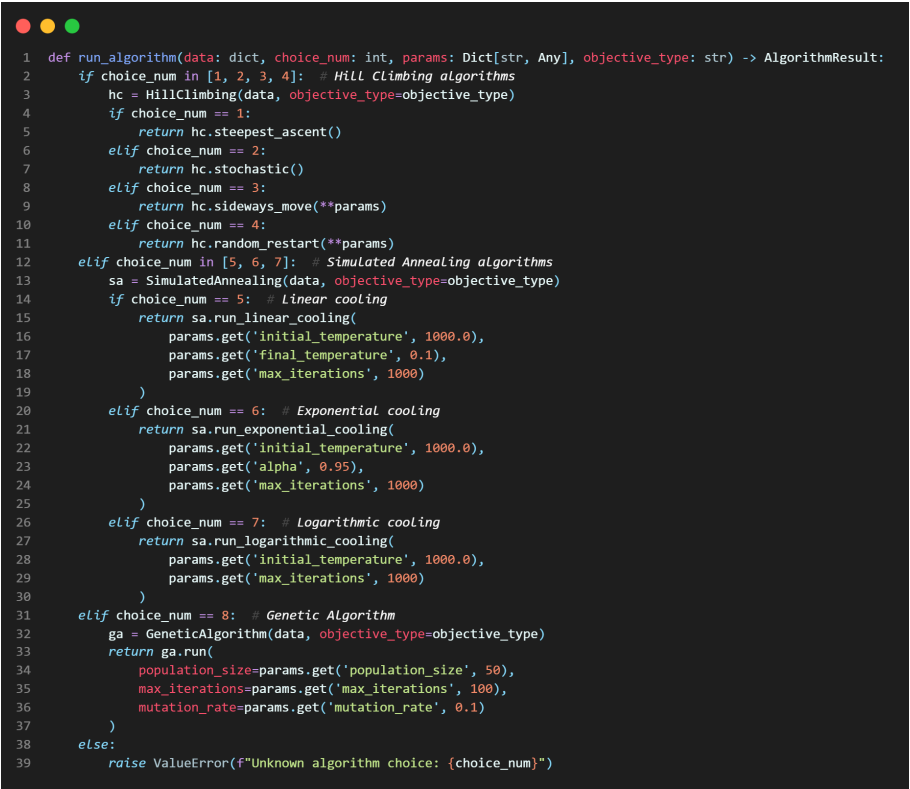
Membuat plot analisis faktor probabilitas penerimaan $e^{(\Delta E/T)}$ terhadap iterasi untuk algoritma simulated annealing.

```
1 def plot_simulated_annealing_eET(result, run_num: int):
2     if not hasattr(result, 'objective_history') or not result.objective_history:
3         print("No objective history available")
4         return
5
6     if not hasattr(result, 'temperature_history') or not result.temperature_history:
7         print("No temperature history available")
8         return
9
10    eET_history = []
11    min_len = min(len(result.objective_history), len(result.temperature_history))
12
13    for i in range(1, min_len):
14        delta_E = result.objective_history[i] - result.objective_history[i-1]
15        temperature = result.temperature_history[i]
16
17        if temperature > 0:
18            eET = np.exp(delta_E / temperature)
19        else:
20            eET = 0 if delta_E > 0 else 1
21
22        eET_history.append(eET)
23
24    fig, ax = plt.subplots(3, 1, figsize=(12, 14))
25    fig.suptitle(f'Simulated Annealing - e(ΔE/T) Analysis (Run {run_num})',
26                fontsize=15, fontweight='bold')
27
28    iterations = list(range(1, len(eET_history) + 1))
29
30    ax.plot(iterations, eET_history, 'purple', linewidth=2.5, label='e(ΔE/T)')
31    ax.fill_between(iterations, eET_history, alpha=0.3, color='purple')
32    ax.axhline(y=1, color='red', linestyle='--', linewidth=1, alpha=0.5, label='Threshold (e0 = 1)')
33    ax.set_xlabel('Iteration', fontsize=12)
34    ax.set_ylabel('e(ΔE/T) Value', fontsize=12)
35    ax.set_title('Acceptance Probability Factor e(ΔE/T) Over Iterations', fontsize=13, fontweight='bold')
36    ax.legend(fontsize=10)
37    ax.grid(True, alpha=0.3)
38
39    avg_eET = np.mean(eET_history)
40    max_eET = max(eET_history)
41    min_eET = min(eET_history)
42
43    stats_text = f'Average e(ΔE/T): {avg_eET:.4f}\n'
44    stats_text += f'Max e(ΔE/T): {max_eET:.4f}\n'
45    stats_text += f'Min e(ΔE/T): {min_eET:.4f}'
46
47    ax.text(0.02, 0.98, stats_text, transform=ax[0].transAxes,
48           fontsize=9, verticalalignment='top',
49           bbox=dict(boxstyle='round', facecolor='plum', alpha=0.7))
50
51    plt.tight_layout()
52
53    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
54    current_dir = os.path.dirname(os.path.abspath(__file__))
55    project_root = os.path.dirname(current_dir)
56    plots_dir = os.path.join(project_root, 'plots')
57
58    if not os.path.exists(plots_dir):
59        os.makedirs(plots_dir)
60
61    filename = f"{timestamp}_Simulated_Annealing_eET_run{run_num}.png"
62    save_path = os.path.join(plots_dir, filename)
63
64    plt.savefig(save_path, dpi=300, bbox_inches='tight')
65
66    plt.close()
```

12. Main

Nama Fungsi	Tipe	Parameter
-------------	------	-----------

get_int_input	int	prompt: str, default: int, min_val: int
Deskripsi: Meminta input berupa bilangan bulat dari pengguna dengan validasi nilai minimum dan default. <pre> 1 def get_int_input(prompt: str, default: int, min_val: int = 1) -> int: 2 while True: 3 try: 4 user_input = input(f"{prompt} (default={default}): ").strip() 5 if user_input == "": 6 return default 7 value = int(user_input) 8 if value < min_val: 9 print(f"Nilai harus >= {min_val}. Coba lagi.") 10 continue 11 return value 12 except ValueError: 13 print("Input tidak valid. Masukkan angka.") </pre>		
get_objective_type	str	
Deskripsi: Meminta pengguna memilih tipe fungsi objektif yang akan digunakan (student_conflict, room_conflict, capacity, atau combined). <pre> 1 def get_objective_type() -> str: 2 print("\n" + "="*60) 3 print("Pilih Objective Function:") 4 print("="*60) 5 print("1. Student Conflict") 6 print("2. Room Conflict") 7 print("3. Room Capacity") 8 print("4. Combined (semua)") 9 10 while True: 11 choice = input("Pilihan (1-4, default=4): ").strip() 12 if choice == "" or choice == "4": 13 return "combined" 14 elif choice == "1": 15 return "student_conflict" 16 elif choice == "2": 17 return "room_conflict" 18 elif choice == "3": 19 return "capacity" 20 else: 21 print("Pilihan tidak valid. Coba lagi.") </pre>		
run_algorithm	AlgorithmResult	data: dict, choice_num: int,

		objective function: Dict[str, Any], objective_type: str
Deskripsi: Menjalankan algoritma yang dipilih (Hill Climbing, Simulated Annealing, atau Genetic Algorithm) dengan parameter yang diberikan.		
 <pre> 1 def run_algorithm(data: dict, choice_num: int, params: Dict[str, Any], objective_type: str) -> AlgorithmResult: 2 if choice_num in [1, 2, 3, 4]: # Hill Climbing algorithms 3 hc = HillClimbing(data, objective_type=objective_type) 4 if choice_num == 1: 5 return hc.steepest_ascent() 6 elif choice_num == 2: 7 return hc.stochastic() 8 elif choice_num == 3: 9 return hc.sideways_move(**params) 10 elif choice_num == 4: 11 return hc.random_restart(**params) 12 elif choice_num in [5, 6, 7]: # Simulated Annealing algorithms 13 sa = SimulatedAnnealing(data, objective_type=objective_type) 14 if choice_num == 5: # Linear cooling 15 return sa.run_linear_cooling(16 params.get('initial_temperature', 1000.0), 17 params.get('final_temperature', 0.1), 18 params.get('max_iterations', 1000) 19) 20 elif choice_num == 6: # Exponential cooling 21 return sa.run_exponential_cooling(22 params.get('initial_temperature', 1000.0), 23 params.get('alpha', 0.95), 24 params.get('max_iterations', 1000) 25) 26 elif choice_num == 7: # Logarithmic cooling 27 return sa.run_logarithmic_cooling(28 params.get('initial_temperature', 1000.0), 29 params.get('max_iterations', 1000) 30) 31 elif choice_num == 8: # Genetic Algorithm 32 ga = GeneticAlgorithm(data, objective_type=objective_type) 33 return ga.run(34 population_size=params.get('population_size', 50), 35 max_iterations=params.get('max_iterations', 100), 36 mutation_rate=params.get('mutation_rate', 0.1) 37) 38 else: 39 raise ValueError(f"Unknown algorithm choice: {choice_num}") </pre>		
run_experiment	None	data: dict, algorithm_name: str, choice_num: int, num_runs: int, objective_type: str
Deskripsi: Menjalankan eksperimen untuk algoritma yang dipilih sebanyak num_runs kali, mencetak hasilnya, dan menghasilkan plot.		

```

1 def run_experiment(data: dict, algorithm_name: str, choice_num: int, num_runs: int, objective_type: str):
2     print(f"\n{'#'*60}")
3     print(f"EKSPERIMEN: {algorithm_name}")
4     print(f"{'#'*60}")
5
6     for run in range(num_runs):
7         params = get_algorithm_parameters(choice_num, run + 1)
8         print(f"\n--- Run {run + 1}/{num_runs} ---")
9         result = run_algorithm(data, choice_num, params, objective_type)
10
11         if choice_num == 8:
12             print_genetic_result(result, algorithm_name, run + 1)
13         elif isinstance(result, SimulatedAnnealingResult):
14             print_simulated_annealing_result(result, algorithm_name, run + 1)
15         else:
16             print_hill_climbing_result(result, algorithm_name, run + 1)
17
18         print_schedule(result.initial_state, f"STATE AWAL (Run {run + 1})")
19         print_schedule(result.final_state, f"STATE AKHIR (Run {run + 1})")
20
21         print(f"\n{'='*60}")
22         print("GENERATING PLOTS")
23         print(f"{'='*60}")
24
25     try:
26         plot.plot_objective_history(result, algorithm_name, run + 1)
27
28         if isinstance(result, SimulatedAnnealingResult):
29             plot.plot_simulated_annealing_eT(result, run + 1)
30
31         if choice_num == 8:
32             plot.plot_genetic_population_fitness(result, run + 1)
33
34     except Exception as e:
35         print(f"Error generating plots: {e}")

```

main

int

argv: List[str]

Deskripsi:

Fungsi utama program. Membaca file JSON, meminta input pengguna, menjalankan eksperimen, dan mencetak hasilnya.

```

1 def main(argv: List[str]) -> int:
2     if len(argv) < 2:
3         print("Usage: python -m src.main <input.json>")
4         return 1
5
6     path = argv[1]
7     try:
8         data = parser.load_and_parse(path)
9     except Exception as e:
10        print(f"Failed to read JSON file '{path}': {e}")
11        return 1
12
13    print(f"\nData loaded from: {path}")
14    print_summary(data)
15
16    objective_type = get_objective_type()
17    num_runs = get_int_input("\nJumlah run per algoritma", 3, 1)
18    algorithm_name, choice_num = select_algorithm()
19
20    run_experiment(data, algorithm_name, choice_num, num_runs, objective_type)
21
22    print("\n" + "="*60)
23    print("EKSPERIMEN SELESAI!")
24    print("="*60)
25    return 0
26

```

2.3. Hasil eksperimen dan Analisis

Input file	
input1.json	<pre>{ "kelas_mata_kuliah": [{ "kode": "IF2110_K01", "jumlah_mahasiswa": 80, "sks": 4 }, { "kode": "IF2120_K01", "jumlah_mahasiswa": 75, "sks": 3 }, { "kode": "IF2130_K01", "jumlah_mahasiswa": 60, "sks": 3 }, { "kode": "IF2210_K01", "jumlah_mahasiswa": 90, "sks": 4 }, { "kode": "IF2230_K01", "jumlah_mahasiswa": 55, "sks": 2 }, { "kode": "IF3110_K01", "jumlah_mahasiswa": 70, "sks": 3 }, { "kode": "IF3130_K01", "jumlah_mahasiswa": 65, "sks": 2 }, { "kode": "IF3170_K01", "jumlah_mahasiswa": 50, "sks": 3 }] }</pre>

	<pre> { "kode": "IF3210_K01", "jumlah_mahasiswa": 85, "sks": 4 }, { "kode": "IF3250_K01", "jumlah_mahasiswa": 40, "sks": 2 }, { "kode": "IF4020_K01", "jumlah_mahasiswa": 45, "sks": 3 }, { "kode": "IF4050_K01", "jumlah_mahasiswa": 55, "sks": 2 }], "ruangan": [{ "kode": "7602", "kuota": 50 }, { "kode": "7603", "kuota": 60 }, { "kode": "7604", "kuota": 70 }, { "kode": "7605", "kuota": 80 }, { "kode": "7606", "kuota": 90 }, { "kode": "7610", "kuota": 100 },] </pre>
--	--

```
{
  "kode": "multimedia1",
  "kuota": 40
},
{
  "kode": "multimedia2",
  "kuota": 45
}
],
"mahasiswa": [
  {
    "nim": "13521001",
    "daftar_mk": ["IF2110_K01",
"IF2120_K01", "IF2130_K01"],
    "prioritas": [1, 2, 3]
  },
  {
    "nim": "13521002",
    "daftar_mk": ["IF2210_K01",
"IF2230_K01", "IF2120_K01"],
    "prioritas": [1, 2, 3]
  },
  {
    "nim": "13521003",
    "daftar_mk": ["IF2110_K01",
"IF2210_K01", "IF2230_K01"],
    "prioritas": [1, 2, 3]
  },
  {
    "nim": "13521004",
    "daftar_mk": ["IF2130_K01",
"IF2120_K01"],
    "prioritas": [1, 2]
  },
  {
    "nim": "13522001",
    "daftar_mk": ["IF3110_K01",
"IF3130_K01", "IF3170_K01"],
    "prioritas": [1, 2, 3]
  },
  {
    "nim": "13522002",
    "daftar_mk": ["IF3210_K01",
"IF3250_K01", "IF3130_K01"],
    "prioritas": [1, 2, 3]
  },
]
```

	<pre> { "nim": "13522003", "daftar_mk": ["IF3110_K01", "IF3210_K01", "IF3170_K01"], "prioritas": [1, 2, 3] }, { "nim": "13522004", "daftar_mk": ["IF3130_K01", "IF3250_K01"], "prioritas": [1, 2] }, { "nim": "13522005", "daftar_mk": ["IF3110_K01", "IF3170_K01", "IF3210_K01", "IF3250_K01"], "prioritas": [1, 2, 3, 4] }, { "nim": "13523001", "daftar_mk": ["IF4020_K01", "IF4050_K01"], "prioritas": [1, 2] }, { "nim": "13523002", "daftar_mk": ["IF4020_K01", "IF3210_K01"], "prioritas": [1, 2] }, { "nim": "13523003", "daftar_mk": ["IF4050_K01", "IF3110_K01", "IF3130_K01"], "prioritas": [1, 2, 3] }, { "nim": "13521005", "daftar_mk": ["IF2110_K01", "IF2130_K01", "IF2210_K01", "IF2230_K01"], "prioritas": [1, 2, 3, 4] }, { "nim": "13521006", </pre>
--	--

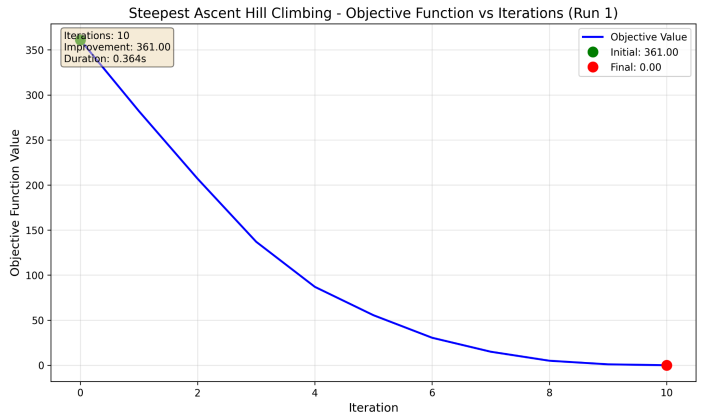
	<pre> "daftar_mk": ["IF2120_K01", "IF2210_K01"], "prioritas": [1, 2] }, { "nim": "13522006", "daftar_mk": ["IF3110_K01", "IF3130_K01", "IF3210_K01"], "prioritas": [1, 2, 3] }] }</pre>
--	--

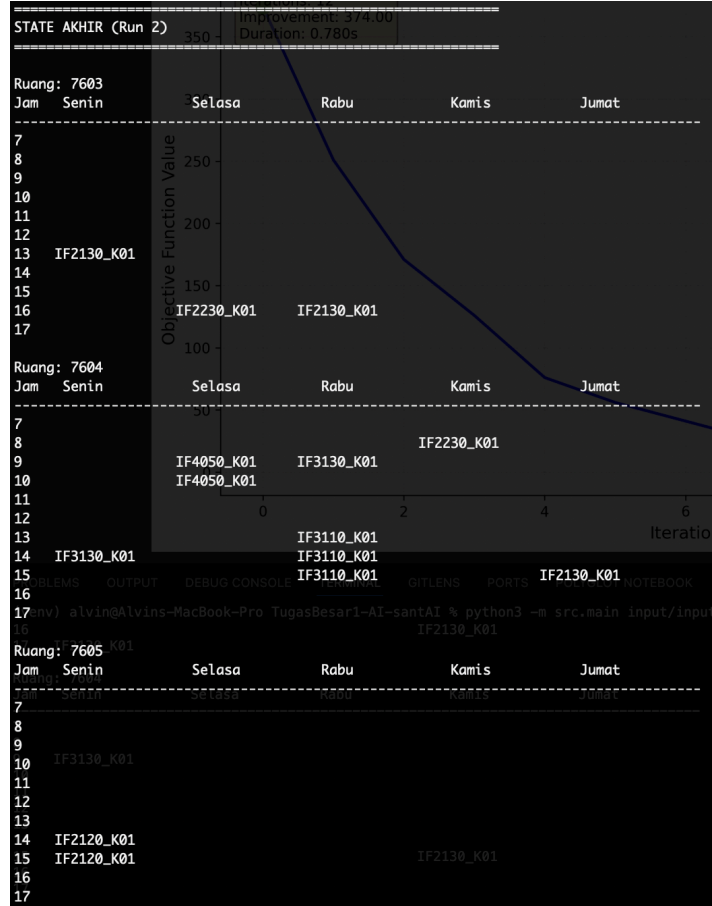
2.3.1. Hill Climbing

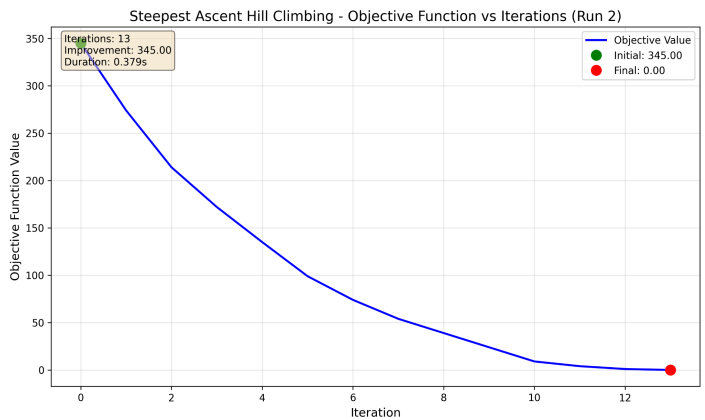
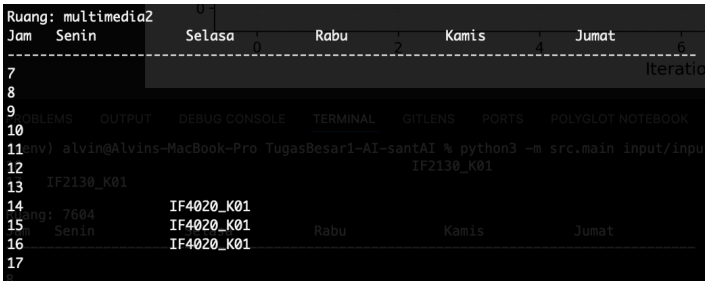
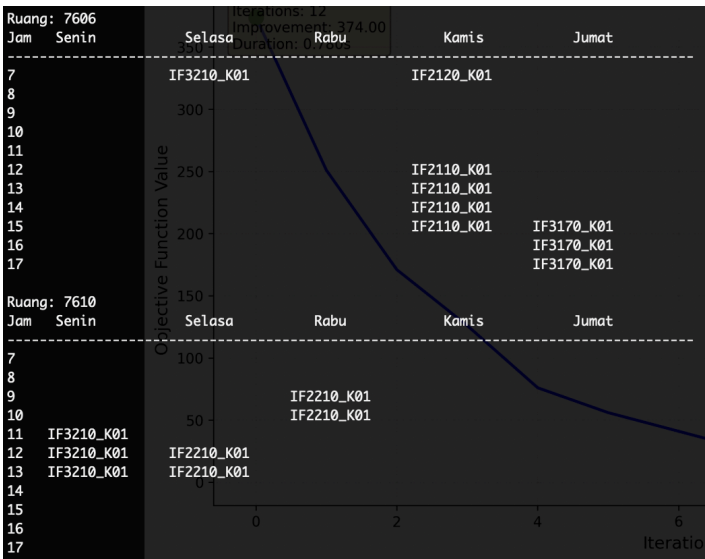
2.3.1.1. Steepest Ascent Hill Climbing

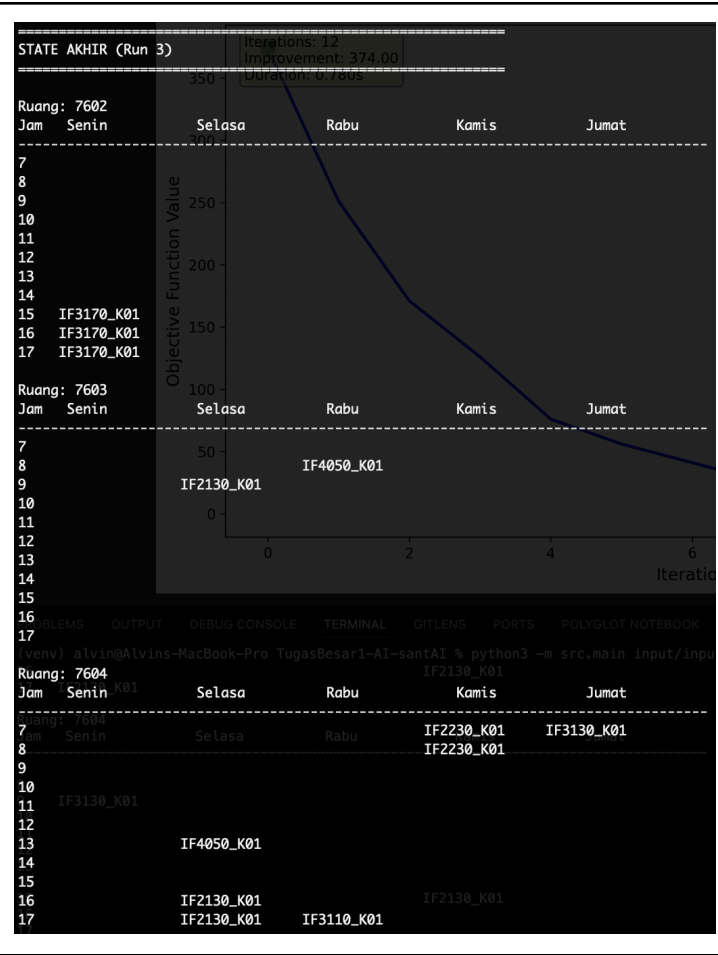
input	Run No	output
-------	--------	--------

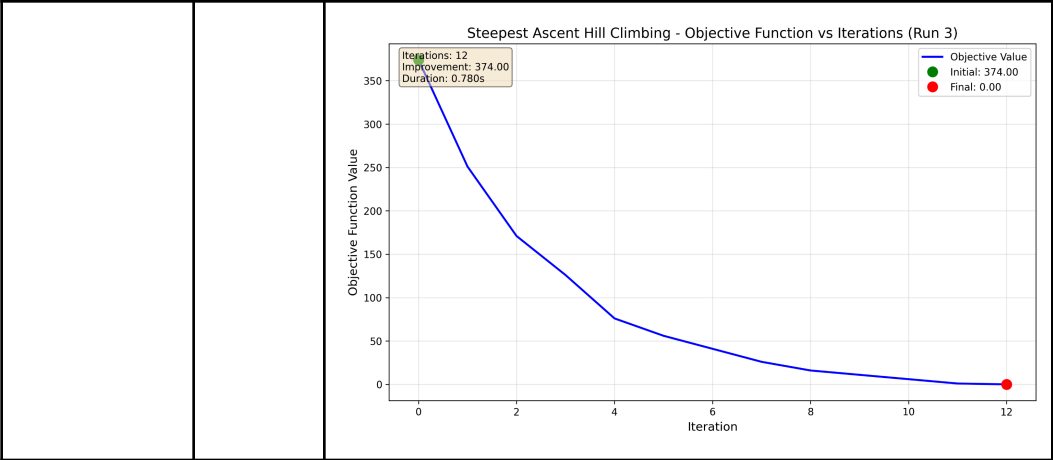
input1.json	1	STATE AKHIR (Run 1) Improvement: 374.00 Duration: 0.780s Ruang: 7602 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 IF3170_K01 16 17 Ruang: 7603 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 Objective Function Value Iteration IF3170_K01 IF3170_K01 IF3170_K01 IF3250_K01 IF3250_K01 IF2230_K01 ITEMS OUTPUT DEBUG CONSOLE TERMINAL GIT LENS PORTS POLYGLOT NOTEBOOK alvin@Alvins-MacBook-Pro TugasBesar1-AI-santAI % python3 -m src.main input/input IF2130_K01 Ruang: 7604 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 IF4050_K01 10 IF3130_K01 11 12 13 14 15 16 IF2130_K01 17
-------------	---	---











2.3.1.2. Stochastic Hill Climbing

input	Run No	output
input1.json	1	<pre>===== Stochastic Hill Climbing (Run 1) - HASIL EKSPERIMEN ===== Nilai Objective Function Awal: 479.00 Nilai Objective Function Akhir: 0.00 Improvement: 479.00 Jumlah Iterasi: 36 Durasi: 2.4726 detik ===== STATE AKHIR (Run 1) ===== Ruangan: 7602 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1041 1042 1043 1044 1045 1046 1047 1048 1049 1050 1051 1052 1053 1054 1055 1056 1057 1058 1059 1060 1061 1062 1063 1064 1065 1066 1067 1068 1069 1070 1071 1072 1073 1074 1075 1076 1077 1078 1079 1080 1081 1082 1083 1084 1085 1086 1087 1088 1089 1090 1091 1092 1093 1094 1095 1096 1097 1098 1099 1100 1101 1102 1103 1104 1105 1106 1107 1108 1109 1110 1111 1112 1113 1114 1115 1116 1117 1118 1119 1120 1121 1122 1123 1124 1125 1126 1127 1128 1129 1130 1131 1132 1133 1134 1135 1136 1137 1138 1139 1140 1141 1142 1143 1144 1145 1146 1147 1148 1149 1150 1151 1152 1153 1154 1155 1156 1157 1158 1159 1160 1161 1162 1163 1164 1165 1166 1167 1168 1169 1170 1171 1172 1173 1174 1175 1176 1177 1178 1179 1180 1181 1182 1183 1184 1185 1186 1187 1188 1189 1190 1191 1192 1193 1194 1195 1196 1197 1198 1199 1200 1201 1202 1203 1204 1205 1206 1207 1208 1209 1210 1211 1212 1213 1214 1215 1216 1217 1218 1219 1220 1221 1222 1223 1224 1225 1226 1227 1228 1229 1230 1231 1232 1233 1234 1235 1236 1237 1238 1239 1240 1241 1242 1243 1244 1245 1246 1247 1248 1249 1250 1251 1252 1253 1254 1255 1256 1257 1258 1259 1260 1261 1262 1263 1264 1265 1266 1267 1268 1269 1270 1271 1272 1273 1274 1275 1276 1277 1278 1279 1280 1281 1282 1283 1284 1285 1286 1287 1288 1289 1290 1291 1292 1293 1294 1295 1296 1297 1298 1299 1300 1301 1302 1303 1304 1305 1306 1307 1308 1309 1310 1311 1312 1313 1314 1315 1316 1317 1318 1319 1320 1321 1322 1323 1324 1325 1326 1327 1328 1329 1330 1331 1332 1333 1334 1335 1336 1337 1338 1339 1340 1341 1342 1343 1344 1345 1346 1347 1348 1349 1350 1351 1352 1353 1354 1355 1356 1357 1358 1359 1360 1361 1362 1363 1364 1365 1366 1367 1368 1369 1370 1371 1372 1373 1374 1375 1376 1377 1378 1379 1380 1381 1382 1383 1384 1385 1386 1387 1388 1389 1390 1391 1392 1393 1394 1395 1396 1397 1398 1399 1400 1401 1402 1403 1404 1405 1406 1407 1408 1409 1410 1411 1412 1413 1414 1415 1416 1417 1418 1419 1420 1421 1422 1423 1424 1425 1426 1427 1428 1429 1430 1431 1432 1433 1434 1435 1436 1437 1438 1439 1440 1441 1442 1443 1444 1445 1446 1447 1448 1449 1450 1451 1452 1453 1454 1455 1456 1457 1458 1459 1460 1461 1462 1463 1464 1465 1466 1467 1468 1469 1470 1471 1472 1473 1474 1475 1476 1477 1478 1479 1480 1481 1482 1483 1484 1485 1486 1487 1488 1489 1490 1491 1492 1493 1494 1495 1496 1497 1498 1499 1500 1501 1502 1503 1504 1505 1506 1507 1508 1509 1510 1511 1512 1513 1514 1515 1516 1517 1518 1519 1520 1521 1522 1523 1524 1525 1526 1527 1528 1529 1530 1531 1532 1533 1534 1535 1536 1537 1538 1539 1540 1541 1542 1543 1544 1545 1546 1547 1548 1549 1550 1551 1552 1553 1554 1555 1556 1557 1558 1559 1560 1561 1562 1563 1564 1565 1566 1567 1568 1569 1570 1571 1572 1573 1574 1575 1576 1577 1578 1579 1580 1581 1582 1583 1584 1585 1586 1587 1588 1589 1590 1591 1592 1593 1594 1595 1596 1597 1598 1599 1600 1601 1602 1603 1604 1605 1606 1607 1608 1609 1610 1611 1612 1613 1614 1615 1616 1617 1618 1619 1620 1621 1622 1623 1624 1625 1626 1627 1628 1629 1630 1631 1632 1633 1634 1635 1636 1637 1638 1639 1640 1641 1642 1643 1644 1645 1646 1647 1648 1649 1650 1651 1652 1653 1654 1655 1656 1657 1658 1659 1660 1661 1662 1663 1664 1665 1666 1667 1668 1669 1670 1671 1672 1673 1674 1675 1676 1677 1678 1679 1680 1681 1682 1683 1684 1685 1686 1687 1688 1689 1690 1691 1692 1693 1694 1695 1696 1697 1698 1699 1700 1701 1702 1703 1704 1705 1706 1707 1708 1709 1710 1711 1712 1713 1714 1715 1716 1717 1718 1719 1720 1721 1722 1723 1724 1725 1726 1727 1728 1729 1730 1731 1732 1733 1734 1735 1736 1737 1738 1739 1740 1741 1742 1743 1744 1745 1746 1747 1748 1749 1750 1751 1752 1753 1754 1755 1756 1757 1758 1759 1760 1761 1762 1763 1764 1765 1766 1767 1768 1769 1770 1771 1772 1773 1774 1775 1776 1777 1778 1779 1780 1781 1782 1783 1784 1785 1786 1787 1788 1789 1790 1791 1792 1793 1794 1795 1796 1797 1798 1799 1800 1801 1802 1803 1804 1805 1806 1807 1808 1809 1810 1811 1812 1813 1814 1815 1816 1817 1818 1819 1820 1821 1822 1823 1824 1825 1826 1827 1828 1829 1830 1831 1832 1833 1834 1835 1836 1837 1838 1839 1840 1841 1842 1843 1844 1845 1846 1847 1848 1849 1850 1851 1852 1853 1854 1855 1856 1857 1858 1859 1860 1861 1862 1863 1864 1865 1866 1867 1868 1869 1870 1871 1872 1873 1874 1875 1876 1877 1878 1879 1880 1881 1882 1883 1884 1885 1886 1887 1888 1889 1890 1891 1892 1893 1894 1895 1896 1897 1898 1899 1900 1901 1902 1903 1904 1905 1906 1907 1908 1909 1910 1911 1912 1913 1914 1915 1916 1917 1918 1919 1920 1921 1922 1923 1924 1925 1926 1927 1928 1929 1930 1931 1932 1933 1934 1935 1936 1937 1938 1939 1940 1941 1942 1943 1944 1945 1946 1947 1948 1949 1950 1951 1952 1953 1954 1955 1956 1957 1958 1959 1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 1970 1971 1972 1973 1974 1975 1976 1977 1978 1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019 2020 2021 2022 2023 2024 2025 2026 2027 2028 2029 2030 2031 2032 2033 2034 2035 2036 2037 2038 2039 2040 2041 2042 2043 2044 2045 2046 2047 2048 2049 2050 2051 2052 2053 2054 2055 2056 2057 2058 2059 2060 2061 2062 2063 2064 2065 2066 2067 2068 2069 2070 2071 2072 2073 2074 2075 2076 2077 2078 2079 2080 2081 2082 2083 2084 2085 2086 2087 2088 2089 2090 2091 2092 2093 2094 2095 2096 2097 2098 2099 2100 2101 2102 2103 2104 2105 2106 2107 2108 2109 2110 2111 2112 2113 2114 2115 2116 2117 2118 2119 2120 2121 2122 2123 2124 2125 2126 2127 2128 2129 2130 2131 2132 2133 2134 2135 2136 2137 2138 2139 2140 2141 2142 2143 2144 2145 2146 2147 2148 2149 2150 2151 2152 2153 2154 2155 2156 2157 2158 2159 2160 2161 2162 2163 2164 2165 2166 2167 2168 2169 2170 2171 2172 2173 2174 2175 2176 2177 2178 2179 2180 2181 2182 2183 2184 2185 2186 2187 2188 2189 2190 2191 2192 2193 2194 2195 2196 2197 2198 2199 2200 2201 2202 2203 2204 2205 2206 2207 2208 2209 2210 2211 2212 2213 2214 2215 2216 2217 2218 2219 2220 2221 2222 2223 2224 2225 2226 2227 2228 2229 2230 2231 2232 2233 2234 2235 2236 2237 2238 2239 2240 2241 2242 2243 2244 2245 2246 2247 2248 2249 2250 2251 2252 2253 2254 2255 2256 2257 2258 2259 2260 2261 2262 2263 2264 2265 2266 2267 2268 2269 2270 2271 2272 2273 2274 2275 2276 2277 2278 2279 2280 2281 2282 2283 2284 2285 2286 2287 2288 2289 2290 2291 2292 2293 2294 2295 2296 2297 2298 2299 2300 2301 2302 2303 2304 2305 2306 2307 2308 2309 2310 2311 2312 2313 2314 2315 2316 2317 2318 2319 2320 2321 2322 2323 2324 2325 2326 2327 2328 2329 2330 2331 2332 2333 2334 2335 2336 2337 2338 2339 2340 2341 2342 2343 2344 2345 2346 2347 2348 2349 2350 2351 2352 2353 2354 2355 2356 2357 2358 2359 2360 2361 2362 2363 2364 2365 2366 2367 2368 2369 2370 2371 2372 2373 2374 2375 2376 2377 2378 2379 2380 2381 2382 2383 2384 2385 2386 2387 2388 2389 2390 2391 2392 2393 2394 2395 2396 2397 2398 2399 2400 2401 2402 2403 2404 2405 2406 2407 2408 2409 2410 2411 2412 2413 2414 2415 2416 2417 2418 2419 2420 2421 2422 2423 2424 2425 2426 2427 2428 2429 2430 2431 2432 2433 2434 2435 2436 2437 2438 2439 2440 2441 2442 2443 2444 2445 2446 2447 2448 2449 2450 2451 2452 2453 2454 2455 2456 2457 2458 2459 2460 2461 2462 2463 2464 2465 2466 2467 2468 2469 2470 2471 2472 2473 2474 2475 2476 2477 2478 2479 2480 2481 2482 2483 2484 2485 2486 2487 2488 2489 2490 2491 2492 2493 2494 2495 2496 2497 2498 2499 2500 2501 2502 2503 2504 2505 2506 2507 2508 2509 2510 2511 2512 2513 2514 2515 2516 2517 2518 2519 2520 2521 2522 2523 2524 2525 2526 2527 2528 2529 2530 2531 2532 2533 2534 2535 2536 2537 2538 2539 2540 2541 2542 2543 2544 2545 2546 2547 2548 2549 2550 2551 2552 2553 2554 2555 2556 2557 2558 2559 2560 2561 2562 2563 2564 2565 2566 2567 2568 2569 2570 2571 2572 2573 2574 2575 2576 2577 2578 2579 2580 2581 2582 2583 2584 2585 2586 2587 2588 2589 2590 2591 2592 2593 2594 25</pre>

		Ruang: 7605 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 IF2110_K01 IF2110_K01 Ruang: 7606 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 IF2230_K01 IF2230_K01 Ruang: 7610 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 IF2210_K01 Ruang: multimedia2 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 IF3250_K01 IF4020_K01 IF4020_K01 IF4020_K01
--	--	---

Ruang: 7696

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7	IF3170_K01		IF2210_K01		
8	IF3170_K01		IF2210_K01		
9	IF3170_K01		IF2210_K01		
10					
11			IF2110_K01		
12	IF2110_K01				
13					
14		IF3210_K01	IF2210_K01		
15		IF3210_K01			
16		IF3210_K01			
17		IF2120_K01			

Ruang: 7610

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8	IF4020_K01				IF2130_K01
9					IF2130_K01
10					IF2130_K01
11			IF2110_K01		
12					
13					
14					
15			IF3130_K01		
16			IF3130_K01		
17	IF3210_K01				

Ruang: multimedia1

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9					
10			IF3250_K01		
11			IF3250_K01		
12					
13					
14					
15					
16					
17					

Ruang: multimedia2

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					IF4020_K01
9					IF4020_K01
10					
11					
12					
13					
14					
15					
16					
17					

3

=====

Stochastic Hill Climbing (Run 3) - HASIL EKSPERIMEN

=====

Nilai Objective Function Awal: 293.00

Nilai Objective Function Akhir: 0.00

Improvement: 293.00

Jumlah Iterasi: 16

Durasi: 1.4856 detik

=====

STATE AKHIR (Run 3)

=====

Ruang: 7602

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9					
10	IF3250_K01				
11				IF3170_K01	
12				IF3170_K01	
13					
14					
15					
16					
17					

Ruang: 7603

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9					
10					
11					
12	IF2130_K01				
13	IF2130_K01				
14					
15					
16					IF4020_K01
17					

Ruang: 7604

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8			IF4020_K01		
9					
10		IF4050_K01			
11		IF4050_K01			
12				IF3170_K01	
13					
14					
15					
16					
17					

		<pre> Ruang: 7685 Jan Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 IF2130_K01 IF2110_K01 12 IF2110_K01 13 IF2110_K01 14 15 IF2120_K01 16 IF2120_K01 17 IF2120_K01 Ruang: 7686 Jan Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 IF3130_K01 13 IF3130_K01 14 15 IF2230_K01 IF2210_K01 16 17 Ruang: 7610 Jan Senin Selasa Rabu Kamis Jumat ----- 7 8 IF3210_K01 IF3210_K01 9 IF3210_K01 10 IF3210_K01 IF2230_K01 11 IF3210_K01 12 IF2210_K01 13 IF2210_K01 14 IF2210_K01 15 IF3110_K01 16 IF3110_K01 17 IF3110_K01 Ruang: multimedia1 Jan Senin Selasa Rabu Kamis Jumat ----- 7 IF3250_K01 8 9 10 11 12 13 14 15 16 17 Ruang: multimedia2 Jan Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 IF4020_K01 12 13 14 15 16 17 </pre>
--	--	---

2.3.1.3. Hill Climbing with Sideways Move

input	Run No	output
input1.json	1	<pre> ===== Hill Climbing with Sideways Move (Run 1) - HASIL EKSPERIMEN ===== Nilai Objective Function Awal: 392.50 Nilai Objective Function Akhir: 0.00 Improvement: 392.50 Jumlah Iterasi: 13 Durasi: 0.9635 detik </pre>

STATE AKHIR (Run 1)

Ruang: 7603

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7					
8				IF4020_K01	
9			IF2130_K01		
10			IF2130_K01		
11			IF2130_K01		
12					
13					
14					
15			IF3170_K01		
16			IF3170_K01		
17			IF3170_K01		

Ruang: 7604

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7	IF4050_K01				
8	IF4050_K01				
9					
10				IF3110_K01	
11					
12	IF3110_K01				
13					
14					
15					
16		IF3130_K01			IF2230_K01
17					

Ruang: 7605

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7					
8					
9					
10					
11					
12			IF2230_K01		
13					
14				IF2120_K01	
15				IF2120_K01	
16					
17					

Ruang: 7605						
Jam	Senin	Selasa	Rabu	Kamis	Jumat	

7						
8						
9						
10						
11						
12			IF2230_K01			
13						
14				IF2120_K01		
15				IF2120_K01		
16						
17						
Ruang: 7606						
Jam	Senin	Selasa	Rabu	Kamis	Jumat	

7						
8	IF3210_K01					
9						
10						
11	IF2120_K01				IF3110_K01	
12						
13						
14						
15	IF3210_K01	IF3210_K01				
16					IF3210_K01	
17						
Ruang: 7610						
Jam	Senin	Selasa	Rabu	Kamis	Jumat	

7						
8						
9						
10					IF2110_K01	
11					IF2110_K01	
12					IF2110_K01	
13					IF2110_K01	
14			IF3130_K01			
15						
16		IF2210_K01				
17		IF2210_K01				

Ruang: multimedia1						
Jam	Senin	Selasa	Rabu	Kamis	Jumat	

7						
8						
9						
10		IF3250_K01				
11						
12						
13						
14						
15						
16						
17						
Ruang: multimedia2						
Jam	Senin	Selasa	Rabu	Kamis	Jumat	

7			IF3250_K01			
8						
9						
10						
11						
12					IF4020_K01	
13					IF4020_K01	
14						
15						
16						
17						

=====

Hill Climbing with Sideways Move (Run 2) - HASIL EKSPERIMEN

=====

Nilai Objective Function Awal: 425.00

Nilai Objective Function Akhir: 0.00

Improvement: 425.00

Jumlah Iterasi: 10

Durasi: 0.5373 detik

=====

STATE AKHIR (Run 2)

=====

Ruang: 7602

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7

8

9

10

11

12

13

14

15

16

17

IF4020_K01

Ruang: 7603

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7

8

9

10

11

12

13

14

15

16

17

IF2130_K01

IF2130_K01

IF2130_K01

IF3170_K01

IF3170_K01

IF3170_K01

Ruang: 7604

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7

8

9

10

11

12

13

14

15

16

17

IF4050_K01

IF4050_K01

IF2230_K01

IF2230_K01

IF3130_K01

IF3130_K01

IF3110_K01

		Ruang: 7605 Jam Senin Selasa Rabu Kamis Jumat 7 8 9 10 11 12 13 14 15 16 17 IF2110_K01 Ruang: 7606 Jam Senin Selasa Rabu Kamis Jumat 7 8 9 10 11 12 13 14 15 16 17 IF3210_K01 IF3210_K01 IF3110_K01 IF4020_K01 IF4020_K01
--	--	--

Ruang: 7610

Jam

Senin

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

IF3210_K01

IF2120_K01

IF2120_K01

IF2210_K01

IF2210_K01

IF2210_K01

Ruang: multimedia2

Jam

Senin

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

IF3250_K01

IF3250_K01

Ruang: 7605					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7				IF2110_K01	
8				IF2110_K01	
9					
10				IF2120_K01	
11					
12					
13	IF4050_K01				
14					
15					
16					
17					
Ruang: 7606					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8					
9		IF2210_K01	IF2110_K01		
10		IF2120_K01	IF2110_K01		
11		IF2120_K01			
12					
13					
14					
15			IF2210_K01		
16					
17					
Ruang: 7610					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8					
9					
10	IF2210_K01		IF3210_K01		
11	IF2210_K01		IF3210_K01		
12	IF3210_K01				
13					
14	IF3110_K01				
15	IF3110_K01				
16	IF3110_K01		IF3210_K01		
17					

Ruang: multimedia1					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

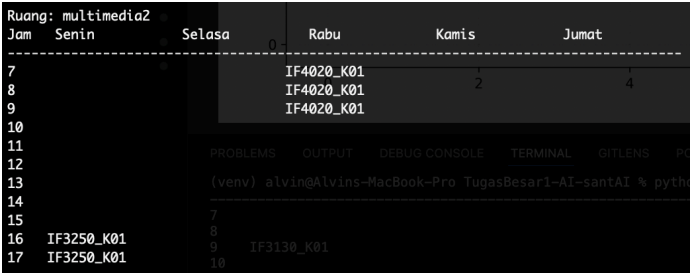
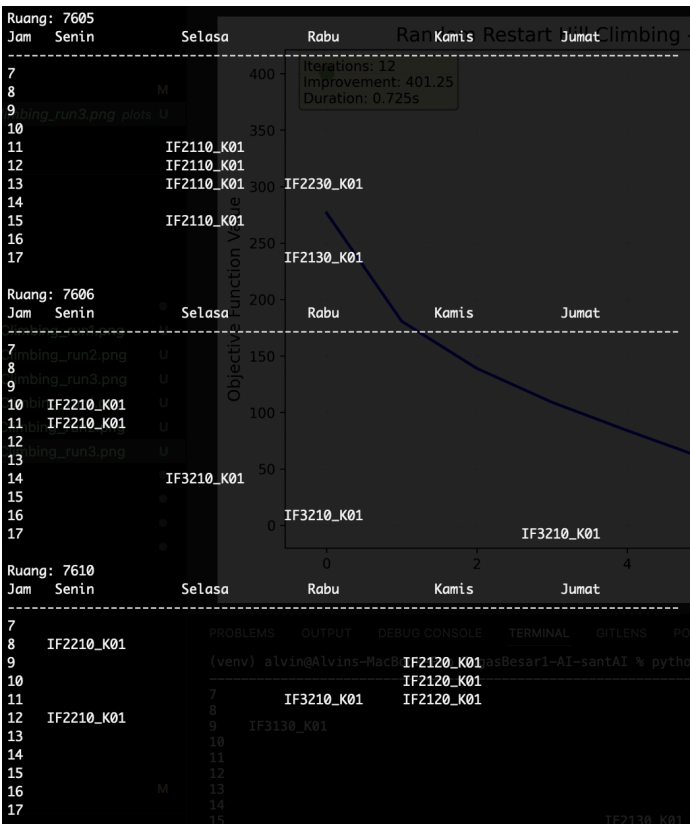
7					
8					
9		IF3250_K01			
10		IF3250_K01			
11					
12					
13					
14					
15					
16					
17					
Ruang: multimedia2					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

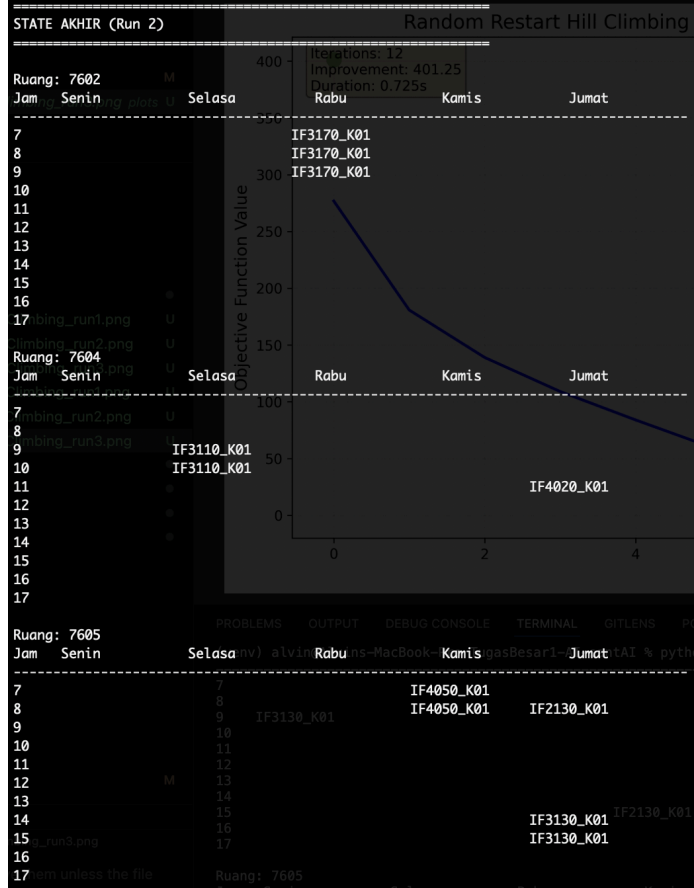
7					
8					
9					
10					
11					IF4020_K01
12					IF4020_K01
13					
14					
15					
16					
17					

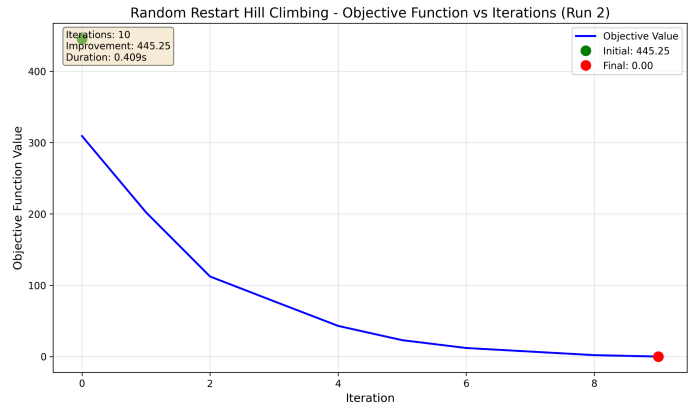
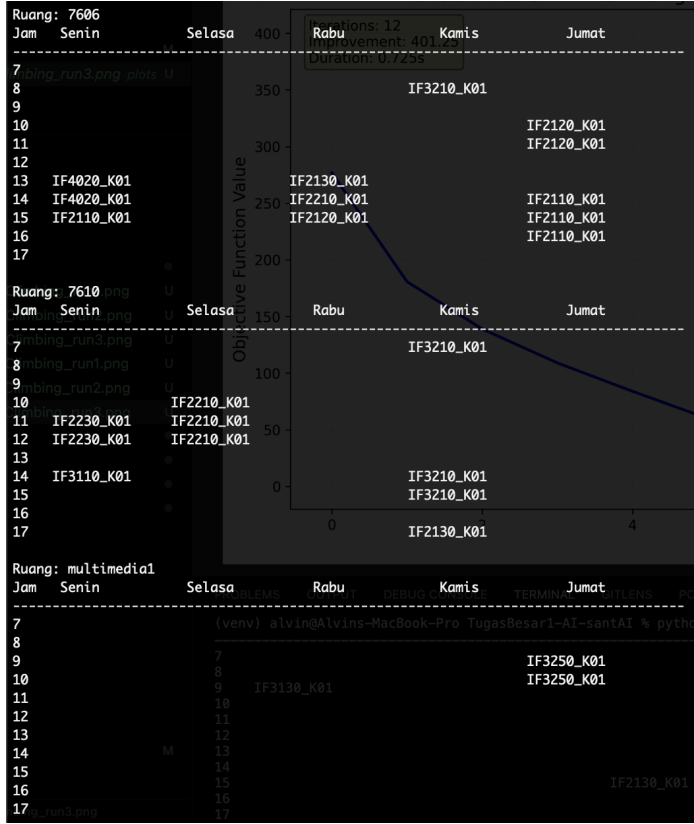
2.3.1.4. Random Restart Hill Climbing

input	Run No	output
-------	--------	--------

input1.json	1	STATE AKHIR (Run 1) Ruang: 7602 Jam: 7602 Senin M 7 8 9 10 11 12 13 14 15 16 17 bing_run1.png bing_run2.png bing_run3.png Ruang: 7603 Jam: 7603 Senin M 7 8 9 10 11 12 13 14 15 16 17 bing_run1.png bing_run2.png bing_run3.png Random Restart Hill Climbing Iterations: 12 Improvement: 401.25 Duration: 0.725s IF3170_K01 IF3170_K01 IF3170_K01 IF4050_K01 IF4050_K01 IF2230_K01 Objective Function Value 400 350 300 250 200 150 100 50 0 Rabu Kamis Jumat PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GIT LENS PO Ruang: 7604 Jam: 7604 Senin M 7 8 9 10 11 12 13 14 15 16 17 IF3110_K01 IF3110_K01 IF3110_K01 IF3130_K01 IF3130_K01 IF3130_K01 IF2130_K01 alvin gasBesari TAI % pyth sim unless the file Ruang: 7605
-------------	---	---







STATE AKHIR (Run 3)		"kelas_mata_kuliah": {				
		"sks": 2				
Ruang: 7602						
Jam	Senin	M	Selasa	Rabu	Kamis	Jumat
7			64	"ruangan": {		
8			65	{		
9			66	"kode": "7602",		
10			67	"kuota": 50		
11			68	},		
12			69	{		
13			70	"kode": "7602IF3170_K01		
14			71	"kuota": 60		
15			72	},		
16			73	IF3170_K01		
17			74	IF4020_K01		
climbing_run1.png	U		75	"kode": "7604",		
Ruang: 7603			76	"kuota": 70		
Jam	Senin	U	Selasa	Rabu	Kamis	Jumat
climbing_run2.png	U					
7	climbing_run1.png	U	77	"kode": "7605",		
8	IF4050_K01	U	78	"kuota": 80		
9			79	IF2230_K01		
10	climbing_run3.png	U	80	{		
11			81	"kode": "7606",		
12			82	"kuota": 90		
13			83	},		
14			84	IF3170_K01		
15			85	{		
16			86	"kode": "7610",		
17			87	"kuota": 100		
			88	},		
Ruang: 7604						
Jam	Senin		PROBLEMS Selasa	OUTPUT Rabu	DEBUG CONSOLE Kamis	TERMINAL Jumat
7						
8			7	IF2130_K01		
9			8			
10			9	IF3130_K01		
11			IF3130_K01	IF3110_K01		
12			IF2130_K01			
13			IF2130_K01			
14			14			
15			15	IF2130_K01		
16			16			
17			17			

Ruang: 7605

Jam

Senin

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

"kelas_mata_kuliah": "SAS 1.2"

"ruangan": {

"kode": "7602",

"kuota": 50

"kode": "7603",

"kuota": 60

IF3110_K01

IF4050_K01

IF3110_K01

Ruang: 7606

Jam

Senin

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

IF3210_K01

IF2210_K01

IF2230_K01

"kode": "7605",

"kuota": 80

"kode": "7606",

"kuota": 90

"kode": "7610",

"kuota": 100

IF3210_K01

IF2110_K01

IF2110_K01

IF2110_K01

IF2110_K01

IF2110_K01

Ruang: 7610

Jam

Senin

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

IF2120_K01

IF2120_K01

IF2120_K01

IF2120_K01

IF2120_K01

IF2120_K01

Ruang: multimedia1

Jam

Senin

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

IF3130_K01

IF3250_K01

IF3250_K01

IF2130_K01

Random Restart Hill Climbing - Objective Function vs Iterations (Run 3)

Iterations: 12

Improvement: 401.25

Duration: 0.725s

Objective Value

Initial: 401.25

Final: 0.00

400

350

300

250

200

150

100

50

0

0

2

4

6

8

10

Iteration

Objective Value

Initial: 401.25

Final: 0.00

2.3.2. Simulated Annealing

2.3.2.1. Simulated Annealing - Linear Cooling

input	Run No	output
-------	--------	--------

input1.json	1	<pre> ===== Simulated Annealing (Linear) (Run 1) - HASIL EKSPERIMEN ===== Nilai Objective Function Awal: 313.00 Nilai Objective Function Akhir: 98.00 Improvement: 215.00 Jumlah Iterasi: 1000 Durasi: 60.3605 detik Temperature Awal: 1000.00 Temperature Akhir: 0.10 Accepted Moves: 966 Rejected Moves: 34 Acceptance Rate: 96.60% ===== STATE AKHIR (Run 1) ===== Ruang: 7602 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 IF2120_K01 14 15 16 17 Ruang: 7603 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 IF3170_K01 12 IF2230_K01 13 IF2230_K01 14 15 16 17 Ruang: 7604 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 IF4050_K01 12 IF4050_K01 13 14 15 16 17 </pre>
-------------	---	--

	Ruang: 7605 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>9</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td>IF3210_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>12</td><td>IF3210_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td>IF2110_K01</td></tr><tr><td>14</td><td></td><td></td><td></td><td>IF3210_K01</td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td>IF3210_K01</td><td></td></tr><tr><td>16</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>17</td><td>IF2210_K01</td><td></td><td></td><td></td><td></td></tr></table> Ruang: 7606 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>9</td><td>IF2110_K01</td><td></td><td></td><td>IF3130_K01</td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td>IF3130_K01</td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td></td><td>IF2110_K01</td></tr><tr><td>16</td><td>IF2210_K01</td><td></td><td>IF2120_K01</td><td></td><td></td></tr><tr><td>17</td><td>IF2210_K01</td><td></td><td>IF2120_K01</td><td></td><td></td></tr></table> Ruang: 7610 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td>IF3110_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td>IF3110_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>9</td><td>IF3110_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>12</td><td></td><td></td><td>IF2110_K01</td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td>IF2130_K01</td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td>IF2130_K01</td><td></td><td></td><td></td></tr><tr><td>17</td><td></td><td>IF2130_K01</td><td></td><td></td><td></td></tr></table> Ruang: multimedia1 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td>IF3250_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td>IF3250_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>9</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td></td><td>IF4020_K01</td><td></td><td></td><td></td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td>IF3170_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td>IF3170_K01</td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td></td><td></td><td>IF4020_K01</td><td></td></tr><tr><td>17</td><td></td><td></td><td></td><td>IF4020_K01</td><td></td></tr></table>	Jam	Senin	Selasa	Rabu	Kamis	Jumat	7						8						9						10						11	IF3210_K01					12	IF3210_K01					13					IF2110_K01	14				IF3210_K01		15				IF3210_K01		16						17	IF2210_K01					Jam	Senin	Selasa	Rabu	Kamis	Jumat	7						8						9	IF2110_K01			IF3130_K01		10				IF3130_K01		11						12						13						14						15					IF2110_K01	16	IF2210_K01		IF2120_K01			17	IF2210_K01		IF2120_K01			Jam	Senin	Selasa	Rabu	Kamis	Jumat	7	IF3110_K01					8	IF3110_K01					9	IF3110_K01					10						11						12			IF2110_K01			13						14						15		IF2130_K01				16		IF2130_K01				17		IF2130_K01				Jam	Senin	Selasa	Rabu	Kamis	Jumat	7	IF3250_K01					8	IF3250_K01					9						10						11		IF4020_K01				12						13	IF3170_K01					14	IF3170_K01					15						16				IF4020_K01		17				IF4020_K01	
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																																																																																																																																																																												
7																																																																																																																																																																																																																																																																																																	
8																																																																																																																																																																																																																																																																																																	
9																																																																																																																																																																																																																																																																																																	
10																																																																																																																																																																																																																																																																																																	
11	IF3210_K01																																																																																																																																																																																																																																																																																																
12	IF3210_K01																																																																																																																																																																																																																																																																																																
13					IF2110_K01																																																																																																																																																																																																																																																																																												
14				IF3210_K01																																																																																																																																																																																																																																																																																													
15				IF3210_K01																																																																																																																																																																																																																																																																																													
16																																																																																																																																																																																																																																																																																																	
17	IF2210_K01																																																																																																																																																																																																																																																																																																
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																																																																																																																																																																												
7																																																																																																																																																																																																																																																																																																	
8																																																																																																																																																																																																																																																																																																	
9	IF2110_K01			IF3130_K01																																																																																																																																																																																																																																																																																													
10				IF3130_K01																																																																																																																																																																																																																																																																																													
11																																																																																																																																																																																																																																																																																																	
12																																																																																																																																																																																																																																																																																																	
13																																																																																																																																																																																																																																																																																																	
14																																																																																																																																																																																																																																																																																																	
15					IF2110_K01																																																																																																																																																																																																																																																																																												
16	IF2210_K01		IF2120_K01																																																																																																																																																																																																																																																																																														
17	IF2210_K01		IF2120_K01																																																																																																																																																																																																																																																																																														
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																																																																																																																																																																												
7	IF3110_K01																																																																																																																																																																																																																																																																																																
8	IF3110_K01																																																																																																																																																																																																																																																																																																
9	IF3110_K01																																																																																																																																																																																																																																																																																																
10																																																																																																																																																																																																																																																																																																	
11																																																																																																																																																																																																																																																																																																	
12			IF2110_K01																																																																																																																																																																																																																																																																																														
13																																																																																																																																																																																																																																																																																																	
14																																																																																																																																																																																																																																																																																																	
15		IF2130_K01																																																																																																																																																																																																																																																																																															
16		IF2130_K01																																																																																																																																																																																																																																																																																															
17		IF2130_K01																																																																																																																																																																																																																																																																																															
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																																																																																																																																																																												
7	IF3250_K01																																																																																																																																																																																																																																																																																																
8	IF3250_K01																																																																																																																																																																																																																																																																																																
9																																																																																																																																																																																																																																																																																																	
10																																																																																																																																																																																																																																																																																																	
11		IF4020_K01																																																																																																																																																																																																																																																																																															
12																																																																																																																																																																																																																																																																																																	
13	IF3170_K01																																																																																																																																																																																																																																																																																																
14	IF3170_K01																																																																																																																																																																																																																																																																																																
15																																																																																																																																																																																																																																																																																																	
16				IF4020_K01																																																																																																																																																																																																																																																																																													
17				IF4020_K01																																																																																																																																																																																																																																																																																													
2	===== Simulated Annealing (Linear) (Run 2) - HASIL EKSPERIMEN ===== Nilai Objective Function Awal: 488.25 Nilai Objective Function Akhir: 109.00 Improvement: 379.25 Jumlah Iterasi: 1000 Durasi: 57.5556 detik Temperature Awal: 1000.00 Temperature Akhir: 0.10 Accepted Moves: 953 Rejected Moves: 47 Acceptance Rate: 95.30%																																																																																																																																																																																																																																																																																																

STATE AKHIR (Run 2)

Ruang: 7602

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9				IF3170_K01	
10				IF3170_K01	
11				IF3170_K01	
12					IF3250_K01
13					
14		IF3130_K01			
15		IF3130_K01			
16		IF3250_K01			
17					

Ruang: 7604

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9					
10					
11					
12					
13	IF3110_K01				
14	IF3110_K01				
15					
16					
17					

Ruang: 7605

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9					
10	IF2110_K01				
11	IF2110_K01				
12	IF2130_K01				
13					
14					
15			IF2110_K01		
16			IF2110_K01		
17					

Ruang: 7606

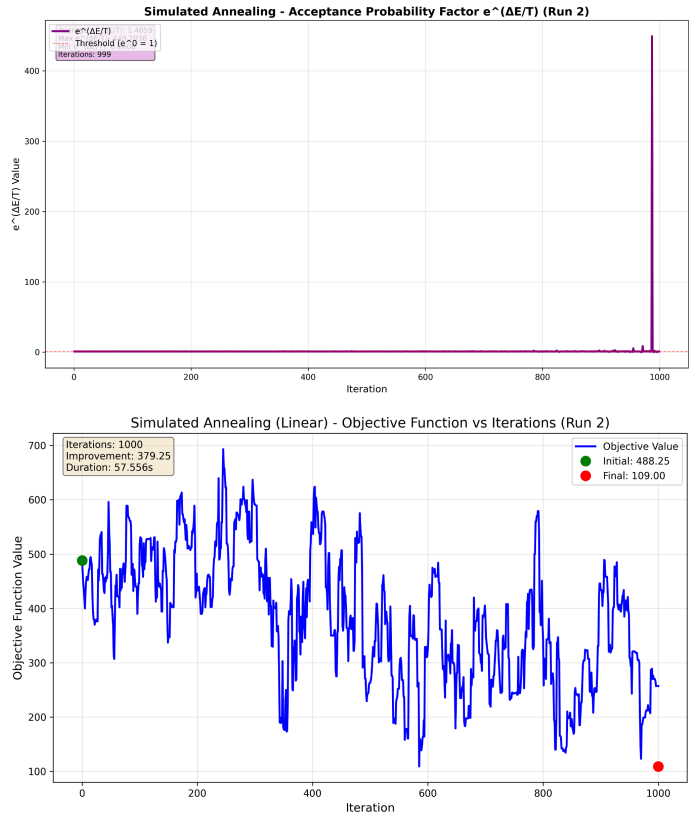
Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9	IF4050_K01				
10					
11		IF2120_K01			
12		IF2120_K01			
13		IF2120_K01			
14					
15					IF2130_K01
16					
17					

Ruang: 7610

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					IF4020_K01
9					
10			IF2230_K01		
11			IF2230_K01		
12					
13			IF2210_K01	IF3210_K01	
14		IF2210_K01	IF2210_K01	IF3210_K01	
15		IF2210_K01		IF3210_K01	
16				IF3210_K01	
17					

Ruang: multimedia1					
Jam	Senin	Selasa	Rabu	Kamis	Jumat
7				IF3110_K01	
8					
9					
10					
11					
12					
13					
14					
15				IF4050_K01	
16					
17					

Ruang: multimedia2					
Jam	Senin	Selasa	Rabu	Kamis	Jumat
7	IF2130_K01				
8					
9					
10					
11					
12					
13					
14					
15					
16					
17					



=====

Simulated Annealing (Linear) (Run 3) - HASIL EKSPERIMEN

=====

Nilai Objective Function Awal: 232.00
Nilai Objective Function Akhir: 105.00
Improvement: 127.00
Jumlah Iterasi: 1000
Durasi: 58.7229 detik
Temperature Awal: 1000.00
Temperature Akhir: 0.10
Accepted Moves: 950
Rejected Moves: 50
Acceptance Rate: 95.00%

=====

STATE AKHIR (Run 3)

=====

Ruang: 7602

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7

8 IF4050_K01

9 IF4050_K01

10

IF2120_K01

11

12

13

14

15

16

17

IF2230_K01

Ruang: 7604

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7

IF3210_K01

8

9

10

11

12

IF3130_K01

13

14

15

IF2130_K01

16

17

Ruang: 7605

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7

IF3110_K01

8

9

10

11

12

13

14

15

16

17

Ruang: 7606					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8					
9					
10					
11			IF2120_K01		
12			IF2120_K01		
13			IF3170_K01		
14	IF3210_K01				
15	IF3210_K01				
16	IF3210_K01				
17					

Ruang: 7610					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

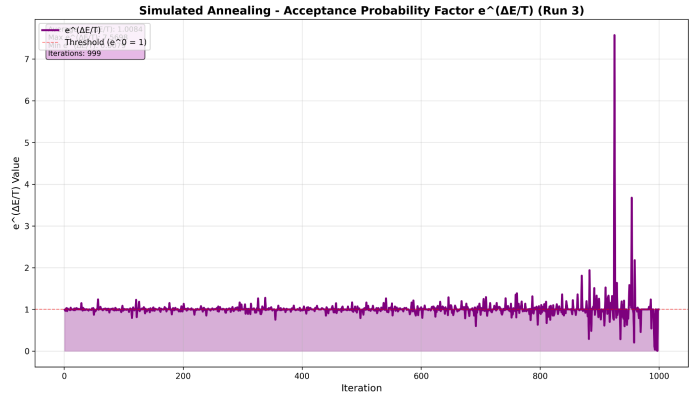
7					
8					
9		IF2230_K01			
10	IF2110_K01	IF2110_K01			
11					IF2210_K01
12					IF2210_K01
13		IF3110_K01			IF2210_K01
14		IF3110_K01			IF2210_K01
15					
16	IF2110_K01				
17	IF2110_K01		IF3130_K01		

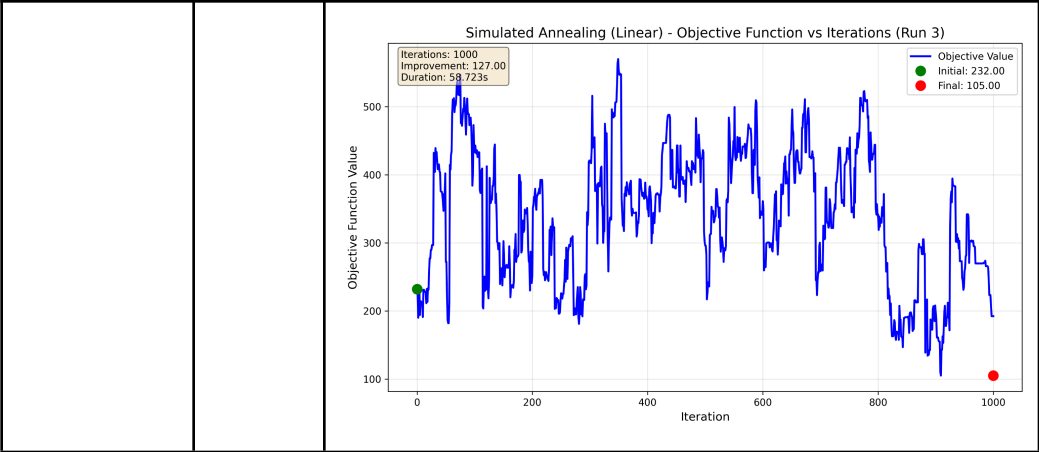
Ruang: multimedia1					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8					
9			IF3250_K01		
10			IF3250_K01		
11					
12					
13					
14					
15					
16					
17					

Ruang: multimedia2					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8					
9					
10					
11					
12		IF4020_K01			
13		IF4020_K01			
14		IF4020_K01			
15					
16				IF2130_K01	
17				IF2130_K01	





2.3.2.2. Simulated Annealing - Exponential Cooling

input	Run No	output
input1.json	1	STATE AKHIR (Run 1) by (Working Tree) src 7602 7603 7604 Senin Senin Senin src src src Selasa Selasa Selasa Rabu Rabu Rabu Kamis Kamis Kamis Jumat Jumat Jumat 7 8 9 10 11 12 13 14 15 16 17 simulated_annealing.py 20251019_221248_Simulated_Annealing_Exponential_run2.png 20251019_221257_Simulated_Annealing_Exponential_run3.png Preview README.md output_formatter.py TUGASBESAR1-AI-SANTAI plots 20251019_215317_Genetic_Algorithm_run2.png 20251019_215640_Genetic_Algorithm_run3.png 20251019_215641_Genetic_Algorithm_run3.png 20251019_215955_Simulated_Annealing_Linear_run1.png 20251019_215956_Simulated_Annealing_Linear_run2.png 20251019_215956_Simulated_Annealing_Linear_run3.png 20251019_221134_Simulated_Annealing_Logarithmic_run1.png 20251019_221134_Simulated_Annealing_Logarithmic_run2.png 20251019_221134_Simulated_Annealing_Logarithmic_run3.png 20251019_221239_Simulated_Annealing_Exponential_run1.png 20251019_221239_Simulated_Annealing_Exponential_run2.png 20251019_221239_Simulated_Annealing_Exponential_run3.png 20251019_221257_Simulated_Annealing_Exponential_run3.png IF2120_K01 IF4050_K01 IF4050_K01 IF3130_K01 IF3130_K01 IF2230_K01 IF2230_K01 IF3110_K01 Objective Function Value 50 40 30 20 10 0

Ruang: 7605

main.py

Selasa

Tre

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

README.md

plot.py

simulated_annealing.py

20251019_221248_Simulated_Annealing_Exponential_run2.png

20251019_221257_Simulated_Annealing_Exponential_run3.png

Preview README.md

output_formatter.py

TUGASBESARI-AI-SANTAI

IF2110_K01

Ruang: 7606

20251019_215317_Genetic_Algorithm_run2.png

20251019_215317_Genetic_Algorithm_run3.png

IF4020_K01

IF4020_K01

20251019_215955_Simulated_Annealing_Linear_run1.png

20251019_215955_Simulated_Annealing_eET_run1.png

20251019_221034_Simulated_Annealing_eET_run1.png

IF2210_K01

IF2210_K01

IF2210_K01

IF2210_K01

IF2210_K01

20251019_221239_Simulated_Annealing_eET_run1.png

20251019_221239_Simulated_Annealing_Exponential_run1.png

20251019_221248_Simulated_Annealing_Exponential_run2.png

Jam

Senin

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

20251019_221257_Simulated_Annealing_Exponential_run3.png

src

__pycache__

IF3210_K01

IF3210_K01

__pycache__

genetic.py

hill_climbing.py

simulated_annealing.py

IF3170_K01

Ruang: multimedia2

plots

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

20251019_215317_Genetic_Algorithm_run2.png

IF3250_K01

IF3250_K01

20251019_215641_Genetic_Algorithm_run3.png

20251019_215955_Simulated_Annealing_Linear_run1.png

20251019_215955_Simulated_Annealing_eET_run1.png

20251019_221034_Simulated_Annealing_eET_run1.png

20251019_221034_Simulated_Annealing_Exponential_run1.png

20251019_221134_Simulated_Annealing_eET_run1.png

20251019_221134_Simulated_Annealing_Logarithmic_run1.png

Objective Function Value

30

20

10

Ruang: multimedia2

plots

Selasa

Rabu

Kamis

Jumat

7

8

9

10

11

12

13

14

15

16

17

20251019_215317_Genetic_Algorithm_run2.png

IF3250_K01

IF3250_K01

20251019_215641_Genetic_Algorithm_run3.png

20251019_215955_Simulated_Annealing_Linear_run1.png

20251019_215955_Simulated_Annealing_eET_run1.png

20251019_221034_Simulated_Annealing_eET_run1.png

20251019_221034_Simulated_Annealing_Exponential_run1.png

20251019_221134_Simulated_Annealing_eET_run1.png

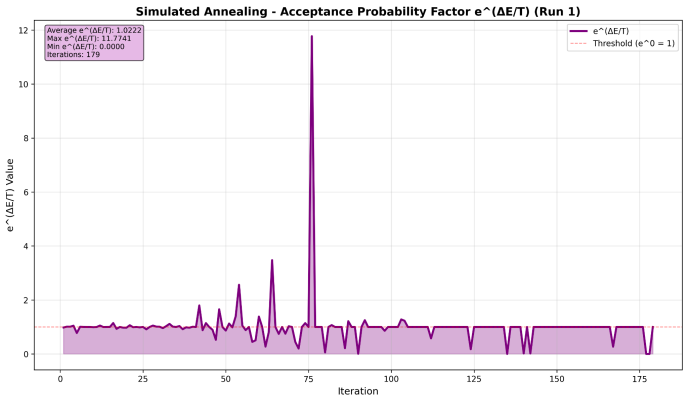
20251019_221134_Simulated_Annealing_Logarithmic_run1.png

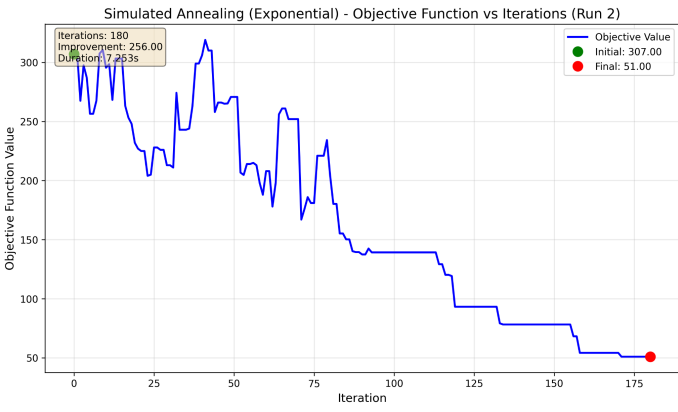
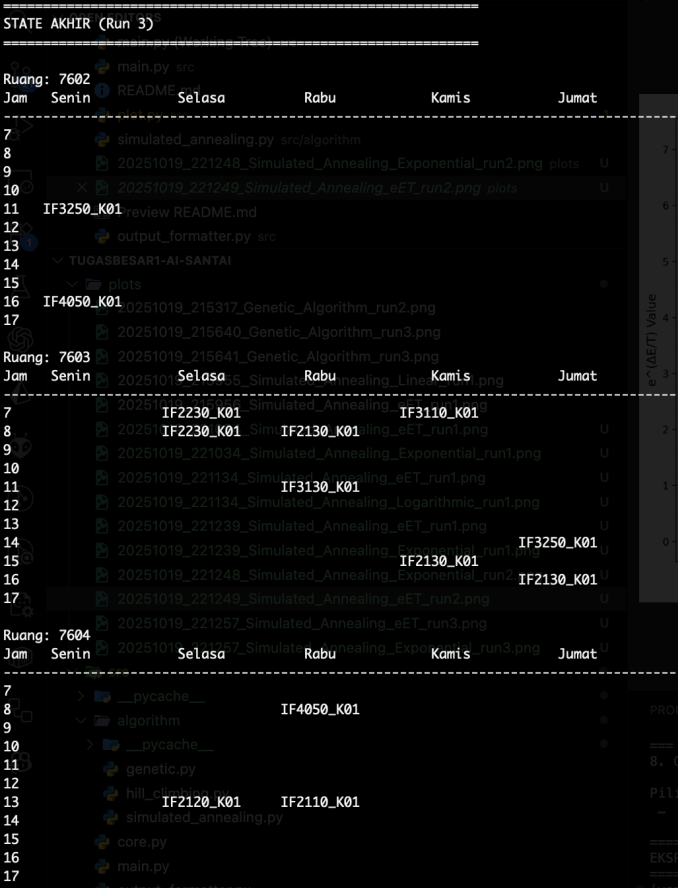
Objective Function Value

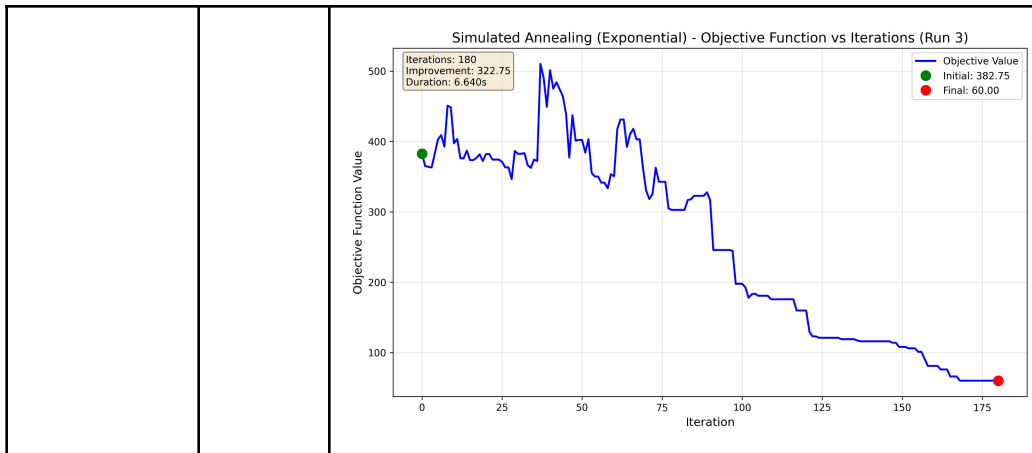
30

20

10



		Simulated Annealing (Exponential) - Objective Function vs Iterations (Run 2) 
	3	STATE AKHIR (Run 3) 



2.3.2.3. Simulated Annealing - Logarithmic Cooling

input	Run No	output
input1.json	1	<pre> ===== Simulated Annealing (Logarithmic) (Run 1) - HASIL EKSPERIMEN ===== Nilai Objective Function Awal: 392.25 Nilai Objective Function Akhir: 151.00 Improvement: 241.25 Jumlah Iterasi: 1000 Durasi: 71.7575 detik Temperature Awal: 1000.00 Temperature Akhir: 0.10 Accepted Moves: 949 Rejected Moves: 51 Acceptance Rate: 94.90% </pre>

=====

STATE AKHIR (Run 1)

=====

Ruang: 7602

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9					
10					
11					
12					
13					
14					
15					
16					
17	IF3250_K01	IF2230_K01			

Ruang: 7603

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9					IF3170_K01
10					IF3170_K01
11					IF3170_K01
12					
13					
14					
15					
16		IF2130_K01			
17					

Ruang: 7604

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7					
8					
9					
10					IF3210_K01
11		IF3130_K01			
12					
13					
14					
15					
16					
17	IF2130_K01	IF3250_K01			IF2110_K01

Ruang: 7605					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7		IF3130_K01			
8					
9		IF4050_K01			
10		IF4050_K01			
11					
12	IF3210_K01				
13			IF2210_K01		
14			IF2210_K01		
15			IF2210_K01		IF2120_K01
16					IF2120_K01
17					IF2120_K01
Ruang: 7606					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8		IF4020_K01			
9		IF4020_K01			
10		IF4020_K01			
11					
12					
13			IF3110_K01		
14		IF2210_K01			
15					
16					
17					
Ruang: 7610					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7			IF2130_K01		
8					
9		IF3210_K01	IF2110_K01		
10			IF2110_K01		
11			IF2110_K01		
12					IF3110_K01
13					
14					
15					
16					
17					
Ruang: multimedia1					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8					
9					
10					
11					
12					
13					
14					
15					IF3210_K01
16					
17					
Ruang: multimedia2					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7	IF3110_K01				
8					
9					
10					
11					
12					
13		IF2230_K01			
14					
15					
16					
17					

```
=====
Simulated Annealing (Logarithmic) (Run 2) - HASIL EKSPERIMEN
=====
```

```

Nilai Objective Function Awal: 362.25
Nilai Objective Function Akhir: 80.25
Improvement: 282.00
Jumlah Iterasi: 1000
Durasi: 63.8063 detik
Temperature Awal: 1000.00
Temperature Akhir: 0.10
Accepted Moves: 943
Rejected Moves: 57
Acceptance Rate: 94.30%

```

```
=====
STATE AKHIR (Run 2)
=====
```

```

Ruang: 7602
Jam  Senin      Selasa      Rabu      Kamis      Jumat
-----
7
8
9          IF2230_K01
10
11
12
13
14                      IF3170_K01
15
16
17

Ruang: 7603
Jam  Senin      Selasa      Rabu      Kamis      Jumat
-----
7
8
9                      IF2130_K01
10                     IF2130_K01
11                     IF2130_K01
12
13
14
15          IF3170_K01
16          IF3170_K01
17

Ruang: 7604
Jam  Senin      Selasa      Rabu      Kamis      Jumat
-----
7
8          IF4050_K01
9          IF4050_K01
10
11
12
13          IF3110_K01
14
15          IF3250_K01      IF3210_K01      IF2120_K01
16                      IF2120_K01
17                      IF2120_K01

```

	Ruang: 7605 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td>IF2110_K01</td></tr><tr><td>9</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td>IF3130_K01</td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td>IF3130_K01</td><td></td><td></td></tr><tr><td>15</td><td></td><td>IF3210_K01</td><td>IF2110_K01</td><td>IF2230_K01</td><td></td></tr><tr><td>16</td><td></td><td></td><td>IF2110_K01</td><td></td><td></td></tr><tr><td>17</td><td>IF3210_K01</td><td></td><td></td><td></td><td>IF2110_K01</td></tr></table> Ruang: 7606 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td>IF3110_K01</td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td>IF3110_K01</td></tr><tr><td>9</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td>IF2210_K01</td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>17</td><td></td><td></td><td></td><td></td><td></td></tr></table> Ruang: 7610 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>9</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td>IF2210_K01</td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td>IF2210_K01</td><td></td></tr><tr><td>12</td><td></td><td></td><td></td><td>IF2210_K01</td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>17</td><td></td><td></td><td></td><td></td><td></td></tr></table> Ruang: multimedia1 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>9</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td>IF4020_K01</td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td>IF4020_K01</td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td>IF4020_K01</td><td></td><td></td><td></td></tr><tr><td>17</td><td></td><td></td><td></td><td></td><td></td></tr></table>	Jam	Senin	Selasa	Rabu	Kamis	Jumat	7						8					IF2110_K01	9						10						11						12						13			IF3130_K01			14			IF3130_K01			15		IF3210_K01	IF2110_K01	IF2230_K01		16			IF2110_K01			17	IF3210_K01				IF2110_K01	Jam	Senin	Selasa	Rabu	Kamis	Jumat	7					IF3110_K01	8					IF3110_K01	9						10						11						12						13		IF2210_K01				14						15						16						17						Jam	Senin	Selasa	Rabu	Kamis	Jumat	7						8						9						10				IF2210_K01		11				IF2210_K01		12				IF2210_K01		13						14						15						16						17						Jam	Senin	Selasa	Rabu	Kamis	Jumat	7						8						9						10						11						12						13						14		IF4020_K01				15		IF4020_K01				16		IF4020_K01				17						3	===== Simulated Annealing (Logarithmic) (Run 3) - HASIL EKSPERIMEN ===== Nilai Objective Function Awal: 337.75 Nilai Objective Function Akhir: 98.00 Improvement: 239.75 Jumlah Iterasi: 1000 Durasi: 72.4421 detik Temperature Awal: 1000.00 Temperature Akhir: 0.10 Accepted Moves: 944 Rejected Moves: 56 Acceptance Rate: 94.40%
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																																																																																																																																																																														
7																																																																																																																																																																																																																																																																																																			
8					IF2110_K01																																																																																																																																																																																																																																																																																														
9																																																																																																																																																																																																																																																																																																			
10																																																																																																																																																																																																																																																																																																			
11																																																																																																																																																																																																																																																																																																			
12																																																																																																																																																																																																																																																																																																			
13			IF3130_K01																																																																																																																																																																																																																																																																																																
14			IF3130_K01																																																																																																																																																																																																																																																																																																
15		IF3210_K01	IF2110_K01	IF2230_K01																																																																																																																																																																																																																																																																																															
16			IF2110_K01																																																																																																																																																																																																																																																																																																
17	IF3210_K01				IF2110_K01																																																																																																																																																																																																																																																																																														
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																																																																																																																																																																														
7					IF3110_K01																																																																																																																																																																																																																																																																																														
8					IF3110_K01																																																																																																																																																																																																																																																																																														
9																																																																																																																																																																																																																																																																																																			
10																																																																																																																																																																																																																																																																																																			
11																																																																																																																																																																																																																																																																																																			
12																																																																																																																																																																																																																																																																																																			
13		IF2210_K01																																																																																																																																																																																																																																																																																																	
14																																																																																																																																																																																																																																																																																																			
15																																																																																																																																																																																																																																																																																																			
16																																																																																																																																																																																																																																																																																																			
17																																																																																																																																																																																																																																																																																																			
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																																																																																																																																																																														
7																																																																																																																																																																																																																																																																																																			
8																																																																																																																																																																																																																																																																																																			
9																																																																																																																																																																																																																																																																																																			
10				IF2210_K01																																																																																																																																																																																																																																																																																															
11				IF2210_K01																																																																																																																																																																																																																																																																																															
12				IF2210_K01																																																																																																																																																																																																																																																																																															
13																																																																																																																																																																																																																																																																																																			
14																																																																																																																																																																																																																																																																																																			
15																																																																																																																																																																																																																																																																																																			
16																																																																																																																																																																																																																																																																																																			
17																																																																																																																																																																																																																																																																																																			
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																																																																																																																																																																														
7																																																																																																																																																																																																																																																																																																			
8																																																																																																																																																																																																																																																																																																			
9																																																																																																																																																																																																																																																																																																			
10																																																																																																																																																																																																																																																																																																			
11																																																																																																																																																																																																																																																																																																			
12																																																																																																																																																																																																																																																																																																			
13																																																																																																																																																																																																																																																																																																			
14		IF4020_K01																																																																																																																																																																																																																																																																																																	
15		IF4020_K01																																																																																																																																																																																																																																																																																																	
16		IF4020_K01																																																																																																																																																																																																																																																																																																	
17																																																																																																																																																																																																																																																																																																			

=====

STATE AKHIR (Run 3)

=====

Ruang: 7602

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7 IF2110_K01

8

9

IF3170_K01

10

IF3170_K01

11

IF3170_K01

12

IF4020_K01

13

14

15

16

17

Ruang: 7603

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7

8

9

IF3210_K01

10

11

12

13

14

15

16

17

Ruang: 7604

Jam	Senin	Selasa	Rabu	Kamis	Jumat
-----	-------	--------	------	-------	-------

7

8

9

10

11

12

13

IF3110_K01

14

IF3130_K01

15

IF3130_K01

16

17

		<pre> Ruang: 7605 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 Ruang: 7606 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 Ruang: 7610 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 Ruang: multimedia1 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 Ruang: multimedia2 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 </pre>
--	--	---

2.3.3. Genetic Algorithm

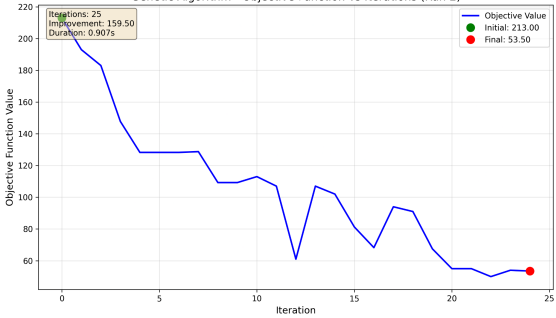
input	Run No	params	output
-------	--------	--------	--------

			Ruang: 7606 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 IF2230_K01 Ruang: 7610 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 IF2210_K01 IF2210_K01 IF2210_K01 IF2210_K01 IF2210_K01 IF2210_K01 IF2210_K01 IF2210_K01 IF2210_K01 IF2210_K01 IF2210_K01 IF3210_K01 IF3110_K01 IF3210_K01 IF3210_K01 IF3210_K01 IF3250_K01
			Genetic Algorithm - Objective Function vs Iterations (Run 1) Genetic Algorithm - Best & Average Fitness vs Iterasi (Run 1)

			Ruang: 7602 Jam Senin Selasa Rabu Kamis Jumat 7 8 9 10 11 12 13 14 15 16 17 IF4050_K01 Ruang: 7603 Jam Senin Selasa Rabu Kamis Jumat 7 8 9 10 11 12 13 14 15 16 17 IF4020_K01 IF3170_K01 IF2230_K01 IF2230_K01 Ruang: 7604 Jam Senin Selasa Rabu Kamis Jumat 7 8 9 10 11 12 13 14 15 16 17 IF2130_K01 IF2130_K01 IF2130_K01 IF2130_K01 Ruang: 7605 Jam Senin Selasa Rabu Kamis Jumat 7 8 9 10 11 12 13 14 15 16 17 IF3170_K01 IF2110_K01 IF2120_K01 IF2120_K01 Ruang: 7606 Jam Senin Selasa Rabu Kamis Jumat 7 8 9 10 11 12 13 14 15 16 17 IF3110_K01 IF3110_K01 IF3210_K01 IF2110_K01 IF3210_K01 IF2110_K01 IF3210_K01 IF2110_K01 IF2210_K01 IF4020_K01 Ruang: 7610 Jam Senin Selasa Rabu Kamis Jumat 7 8 9 10 11 12 13 14 15 16 17 IF2210_K01 IF2210_K01 IF4050_K01
--	--	--	---

			Ruang: multimedia Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 IF2120_K01 Ruang: multimedia2 Jam Senin Selasa Rabu Kamis Jumat ----- 7 8 9 10 11 12 13 14 15 16 17 IF3250_K01 IF3250_K01 IF4020_K01
--	--	--	---

Genetic Algorithm - Objective Function vs Iterations (Run 2)

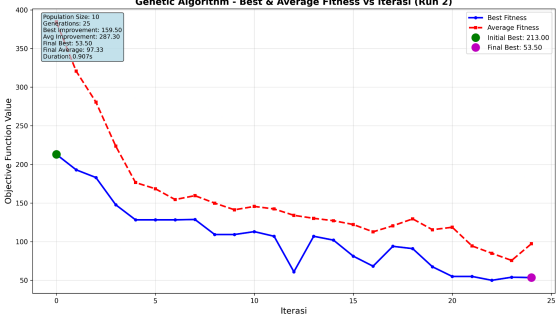


Iterations: 25
Improvement: 159.50
Duration: 0.907s

Objective Value

Initial: 213.00
Final: 53.50

Genetic Algorithm - Best & Average Fitness vs Iterasi (Run 2)

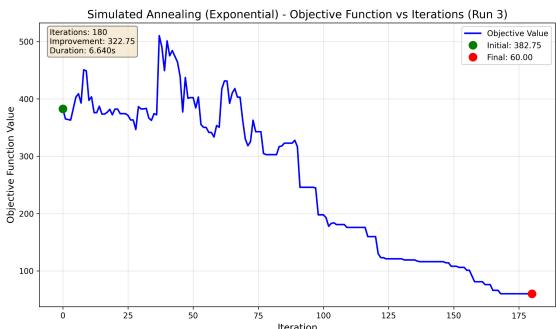
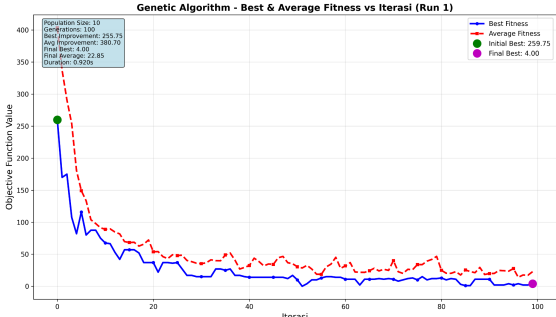


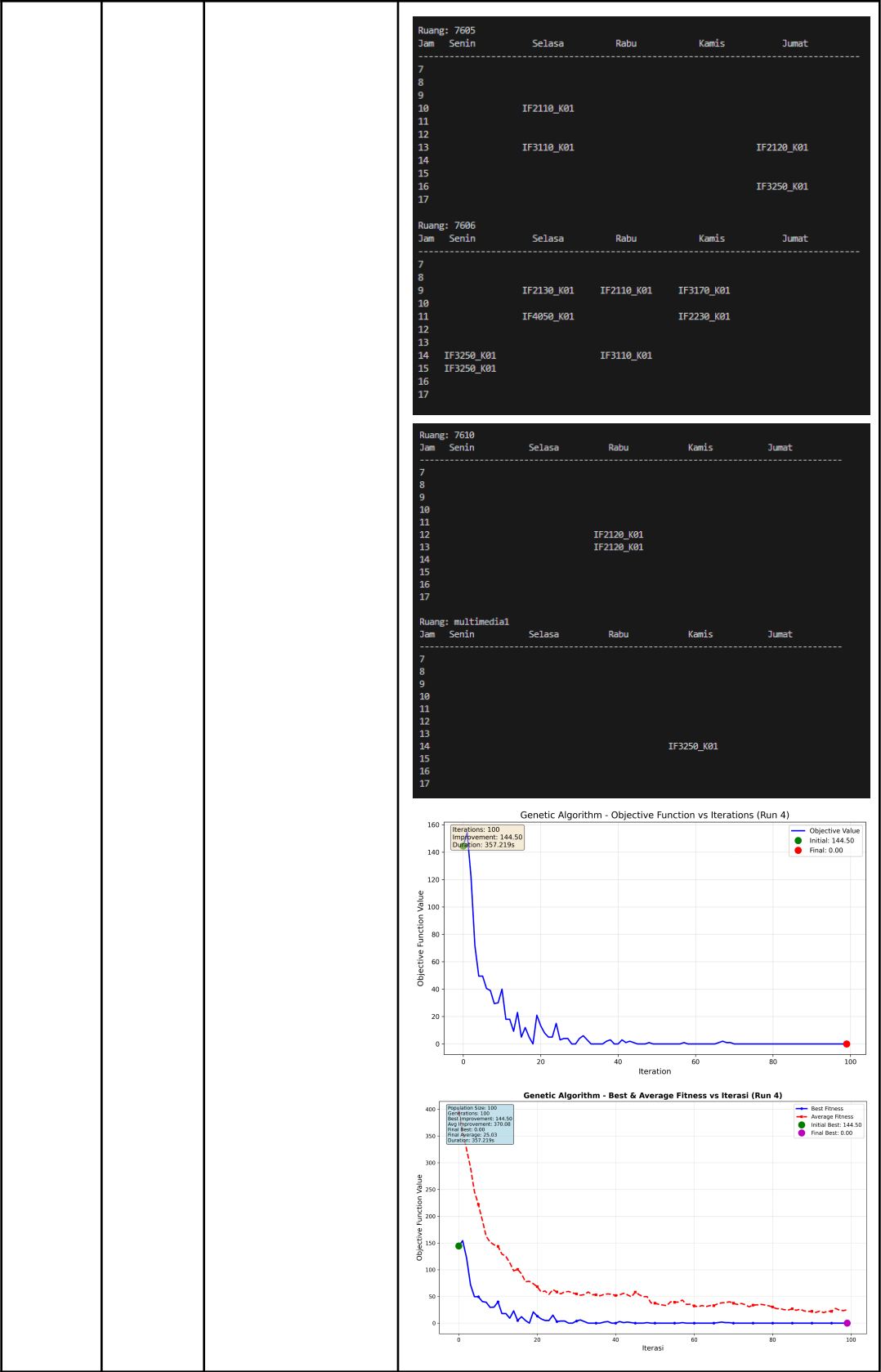
Population Size: 10
Generations: 75
Best Improvement: 159.50
Avg Improvement: 207.30
Final Best: 53.50
Final Average: 22.85
Duration: 0.907s

Objective Function Value

Best Fitness
Average Fitness
Initial Best: 213.00
Final Best: 53.50

| 3 | Population = 10 Iterations = 100 | Genetic Algorithm (Run 1) - HASIL EKSPERIMEN ----- Nilai Objective Function Awal: 259.75 Nilai Objective Function Akhir: 6.00 Improvement: 253.75 Jumlah Iterasi: 100 Durasi: 0.9205 detik Ukuran Populasi: 10 Final Average Fitness: 22.85 |

			Simulated Annealing (Exponential) - Objective Function vs Iterations (Run 3)  Genetic Algorithm - Best & Average Fitness vs Iterasi (Run 1) 																																																																																																																																															
4	Population = 100 Iterations = 100	Nilai Objective Function Awal: 144.50 Nilai Objective Function Akhir: 0.00 Improvement: 144.50 Jumlah Iterasi: 100 Durasi: 357.2189 detik Ukuran Populasi: 100 Final Average Fitness: 25.03 Ruang: 7603 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td>IF3170_K01</td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td>IF3170_K01</td></tr><tr><td>9</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>17</td><td></td><td></td><td></td><td></td><td></td></tr></table> Ruang: 7604 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>9</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td>IF3130_K01</td><td></td></tr><tr><td>11</td><td></td><td>IF2130_K01</td><td></td><td></td><td>IF3110_K01</td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>17</td><td></td><td></td><td></td><td></td><td>IF2230_K01</td></tr></table>	Jam	Senin	Selasa	Rabu	Kamis	Jumat	7					IF3170_K01	8					IF3170_K01	9						10						11						12						13						14						15						16						17						Jam	Senin	Selasa	Rabu	Kamis	Jumat	7						8						9						10				IF3130_K01		11		IF2130_K01			IF3110_K01	12						13						14						15						16						17					IF2230_K01
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																													
7					IF3170_K01																																																																																																																																													
8					IF3170_K01																																																																																																																																													
9																																																																																																																																																		
10																																																																																																																																																		
11																																																																																																																																																		
12																																																																																																																																																		
13																																																																																																																																																		
14																																																																																																																																																		
15																																																																																																																																																		
16																																																																																																																																																		
17																																																																																																																																																		
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																													
7																																																																																																																																																		
8																																																																																																																																																		
9																																																																																																																																																		
10				IF3130_K01																																																																																																																																														
11		IF2130_K01			IF3110_K01																																																																																																																																													
12																																																																																																																																																		
13																																																																																																																																																		
14																																																																																																																																																		
15																																																																																																																																																		
16																																																																																																																																																		
17					IF2230_K01																																																																																																																																													



IF2110_K01

IF3110_K01

IF2120_K01

IF3250_K01

IF2130_K01

IF2110_K01

IF3170_K01

IF4050_K01

IF2230_K01

IF3250_K01

IF3110_K01

IF3250_K01

IF2120_K01

IF2120_K01

IF3250_K01

Genetic Algorithm - Objective Function vs Iterations (Run 4)

Iterations: 100
Improvement: 144.50
Duration: 357.219s

Objective Value

Initial: 144.50
Final: 0.00

Iteration	Objective Value
0	144.50
10	~10
20	~5
30	~2
40	~1
50	~0.5
60	~0.2
70	~0.1
80	~0.05
90	~0.02
100	0.00

Genetic Algorithm - Best & Average Fitness vs Iterasi (Run 4)

Population Size: 100
Generations: 100
Best Improvement: 144.50
Avg Improvement: 270.00
Final Best: 0.00
Final Average: 25.03
Duration: 357.219s

Best Fitness
Average Fitness

Initial Best: 144.50
Final Best: 0.00

Iterasi	Best Fitness	Average Fitness
0	144.50	~150
10	~10	~100
20	~5	~70
30	~2	~50
40	~1	~40
50	~0.5	~35
60	~0.2	~30
70	~0.1	~28
80	~0.05	~26
90	~0.02	~25
100	0.00	~25

5

Population = 60
Iterations = 100

Nilai Objective Function Awal: 130.75
Nilai Objective Function Akhir: 0.00
Improvement: 130.75
Jumlah Iterasi: 100
Durasi: 124.7839 detik
Ukuran Populasi: 60
Final Average Fitness: 28.48

Ruang: 7603					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8	IF2230_K01		IF3170_K01		
9					
10					
11					
12			IF4020_K01		
13			IF4020_K01		
14					IF3250_K01
15					
16					
17					

Ruang: 7604					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8			IF3170_K01		
9					
10					
11					
12					
13	IF3170_K01				
14	IF3170_K01				
15					
16					
17					

Ruang: 7605					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8					
9					
10					
11					
12		IF2130_K01			IF4020_K01
13			IF2130_K01		
14					
15					
16					
17					

Ruang: 7606					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

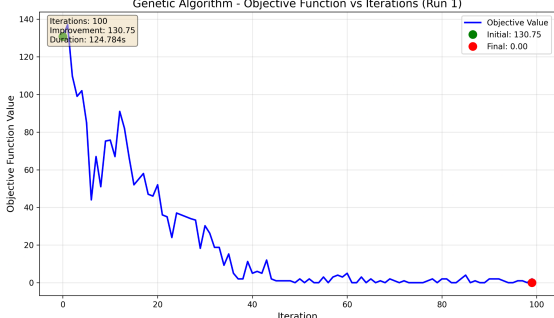
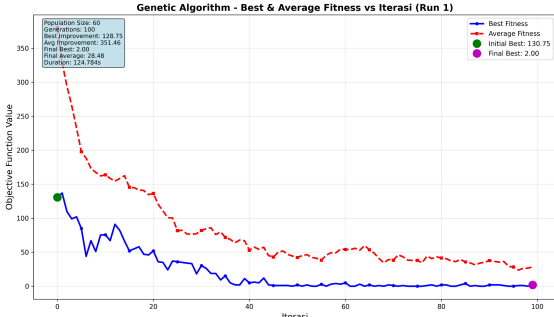
7					IF2230_K01
8					
9				IF2130_K01	
10					
11					
12	IF2230_K01				
13	IF2230_K01				
14					
15					IF3210_K01
16					
17					

Ruang: 7610					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7				IF2130_K01	
8		IF3110_K01			
9					
10					
11					
12					IF2120_K01
13					IF2120_K01
14					IF2120_K01
15			IF2110_K01		
16		IF3210_K01	IF2110_K01		
17			IF2110_K01		

Ruang: multimedia2					
Jam	Senin	Selasa	Rabu	Kamis	Jumat

7					
8					
9					
10					
11					
12					
13					
14					
15				IF3250_K01	
16				IF3250_K01	
17					

			Genetic Algorithm - Objective Function vs Iterations (Run 1)  Iterations: 100 Improvement: 130.75 Duration: 124.784s Legend: Objective Value (blue line), Initial: 130.75 (green dot), Final: 0.00 (red dot) Genetic Algorithm - Best & Average Fitness vs Iterasi (Run 1)  Population Size: 60 Generations: 100 Best Improvement: 128.75 Avg Improvement: 351.46 Final Best: 2.00 Final Average: 28.48 Duration: 114.784s Legend: Best Fitness (blue line), Average Fitness (red dashed line), Initial Best: 130.75 (green dot), Final Best: 2.00 (purple dot)																																																																																																																																															
6	Population = 30 Iterations = 100	Nilai Objective Function Awal: 213.50 Nilai Objective Function Akhir: 4.00 Improvement: 209.50 Jumlah Iterasi: 100 Durasi: 39.1764 detik Ukuran Populasi: 30 Final Average Fitness: 31.24 Ruang: 7603 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td>IF3170_K01</td><td></td><td></td><td></td></tr><tr><td>8</td><td></td><td>IF3170_K01</td><td></td><td></td><td></td></tr><tr><td>9</td><td></td><td>IF3170_K01</td><td></td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td></td><td>IF2130_K01</td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>17</td><td></td><td></td><td></td><td></td><td></td></tr></table> Ruang: 7604 <table><tr><th>Jam</th><th>Senin</th><th>Selasa</th><th>Rabu</th><th>Kamis</th><th>Jumat</th></tr><tr><td>7</td><td></td><td></td><td></td><td></td><td>IF2230_K01</td></tr><tr><td>8</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>9</td><td></td><td></td><td>IF3170_K01</td><td></td><td></td></tr><tr><td>10</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>11</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>12</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>13</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>14</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>15</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>16</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>17</td><td></td><td></td><td>IF4020_K01</td><td></td><td></td></tr></table>	Jam	Senin	Selasa	Rabu	Kamis	Jumat	7		IF3170_K01				8		IF3170_K01				9		IF3170_K01				10						11					IF2130_K01	12						13						14						15						16						17						Jam	Senin	Selasa	Rabu	Kamis	Jumat	7					IF2230_K01	8						9			IF3170_K01			10						11						12						13						14						15						16						17			IF4020_K01		
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																													
7		IF3170_K01																																																																																																																																																
8		IF3170_K01																																																																																																																																																
9		IF3170_K01																																																																																																																																																
10																																																																																																																																																		
11					IF2130_K01																																																																																																																																													
12																																																																																																																																																		
13																																																																																																																																																		
14																																																																																																																																																		
15																																																																																																																																																		
16																																																																																																																																																		
17																																																																																																																																																		
Jam	Senin	Selasa	Rabu	Kamis	Jumat																																																																																																																																													
7					IF2230_K01																																																																																																																																													
8																																																																																																																																																		
9			IF3170_K01																																																																																																																																															
10																																																																																																																																																		
11																																																																																																																																																		
12																																																																																																																																																		
13																																																																																																																																																		
14																																																																																																																																																		
15																																																																																																																																																		
16																																																																																																																																																		
17			IF4020_K01																																																																																																																																															

			Ruang: 7605 Jam Senin Selasa Rabu Kamis Jumat 7 IF4020_K01 IF4020_K01 IF4020_K01 IF3130_K01 IF2230_K01 Ruang: 7606 Jam Senin Selasa Rabu Kamis Jumat 7 IF3250_K01 IF3110_K01 IF3110_K01 IF2120_K01 IF3250_K01 IF4050_K01 IF4050_K01 Ruang: 7610 Jam Senin Selasa Rabu Kamis Jumat 7 IF3210_K01 IF2110_K01 IF2110_K01 IF2110_K01 IF2110_K01 IF2120_K01 IF2130_K01 IF3130_K01 IF3250_K01 IF2120_K01
--	--	--	--

Ruang: multimedia2

Jam

Senin

Selasa

Rabu

Kamis

Jumat

7

IF3250_K01

Genetic Algorithm - Objective Function vs Iterations (Run 2)

Iterations: 100
Improvement: 209.50
Duration: 39.176s

Objective Value

Initial: 213.50

Final: 4.00

Iteration

0

20

40

60

80

100

Objective Function Value

0

50

100

150

200

Genetic Algorithm - Best & Average Fitness vs Iterasi (Run 2)

Population Size: 30
Generations: 100
Best Improvement: 209.50
Avg Improvement: 305.89
Final Best: 3.00
Final Average: 21.24
Duration: 39.176s

Best Fitness

Average Fitness

Initial Best: 213.50

Final Best: 3.00

Iterasi

0

20

40

60

80

100

Objective Function Value

0

100

200

300

400

Analisis

Berdasarkan hasil pengujian terhadap berbagai algoritma optimasi yang diimplementasikan, kemampuan tiap algoritma dalam mendekati global optimum menunjukkan variasi yang cukup signifikan. Pada kelompok algoritma Hill Climbing, pendekatan Steepest Ascent Hill Climbing cenderung cepat mencapai solusi namun mudah terjebak pada local optimum karena selalu memilih langkah terbaik secara lokal tanpa eksplorasi terhadap solusi alternatif. Stochastic Hill Climbing sedikit lebih unggul karena melibatkan unsur acak dalam pemilihan langkah, sehingga kadang mampu keluar dari jebakan local optimum, meskipun kemungkinannya masih terbatas. Hill Climbing dengan Sideways Move juga memperlihatkan peningkatan kinerja pada area plateau dengan kemampuan berpindah ke posisi bernilai sama, namun tetap tidak cukup efektif untuk keluar dari lembah local minima. Sementara itu, Random Restart Hill Climbing memberikan hasil yang paling mendekati global optimum di antara varian Hill Climbing lainnya karena melakukan pencarian berulang dari titik awal yang acak, sehingga peluang menemukan solusi optimal meningkat secara signifikan.

Untuk kelompok Simulated Annealing, hasil pencarian bergantung pada strategi pendinginan (cooling schedule) dan parameter suhu awal (initial temperature). Pada eksperimen ini, ketiga varian, yaitu Linear Cooling, Exponential Cooling, dan Logarithmic Cooling menunjukkan kecenderungan mampu menjelajahi ruang solusi lebih luas dibanding Hill Climbing karena adanya probabilitas penerimaan solusi yang lebih buruk di awal pencarian. Namun, dengan nilai suhu awal default sebesar 1000, algoritma Simulated Annealing pada implementasi ini belum mampu mencapai objective function akhir bernilai 0. Hal ini disebabkan suhu yang menurun terlalu cepat hingga menyentuh batas minimum sebelum algoritma sempat mengeksplorasi ruang solusi secara optimal, sehingga pencarian terhenti pada solusi sub-optimal. Parameter suhu dan laju pendinginan yang terlalu agresif membatasi kemampuan algoritma untuk keluar dari local optimum yang kompleks.

Adapun Genetic Algorithm secara umum menunjukkan performa paling stabil dalam mendekati global optimum. Dengan mekanisme seleksi, crossover, dan mutation, algoritma ini mampu mempertahankan keragaman populasi dan menghindari konvergensi prematur. Meskipun waktu komputasinya lebih lama dibanding Hill Climbing dan Simulated Annealing, Genetic Algorithm berhasil menghasilkan solusi yang konsisten lebih dekat ke optimum global karena eksplorasi ruang solusi dilakukan secara paralel oleh banyak individu. Namun, kualitas hasilnya tetap bergantung pada parameter seperti jumlah populasi dan banyaknya iterasi. Semakin besar nilainya, semakin tinggi kemungkinan mendekati solusi terbaik.

3. Kesimpulan dan Saran

3.1. Kesimpulan

Berdasarkan implementasi dan eksperimen yang telah dilakukan terhadap persoalan penjadwalan kelas mingguan menggunakan berbagai algoritma local search, dapat ditarik beberapa kesimpulan sebagai berikut:

Performa Algoritma Hill Climbing

- Steepest Ascent Hill Climbing cenderung menghasilkan solusi yang lebih baik namun membutuhkan waktu komputasi lebih lama karena harus mengevaluasi semua tetangga di setiap iterasi.
- Stochastic Hill Climbing lebih cepat namun hasilnya bergantung pada faktor keberuntungan dalam pemilihan tetangga secara acak.
- Hill Climbing with Sideways Move membantu mengatasi plateau dengan memperbolehkan perpindahan lateral, meningkatkan peluang menemukan solusi yang lebih optimal.
- Random Restart Hill Climbing menunjukkan performa terbaik di antara varian hill climbing karena mampu keluar dari local optimum melalui restart, meskipun membutuhkan waktu komputasi yang lebih panjang.

Performa Simulated Annealing

- Algoritma ini mampu menerima solusi yang lebih buruk dengan probabilitas tertentu, memungkinkan eksplorasi ruang solusi yang lebih luas dan menghindari local optimum.
- Linear cooling schedule memberikan penurunan temperatur yang stabil dan terprediksi.
- Exponential cooling schedule lebih cepat menuju konvergensi namun berisiko terjebak di local optimum jika penurunan terlalu cepat.
- Logarithmic cooling schedule memberikan eksplorasi lebih lama pada temperatur tinggi, cocok untuk masalah dengan banyak local optimum.

Performa Genetic Algorithm

- Ukuran populasi dan jumlah iterasi sangat mempengaruhi kualitas solusi yang dihasilkan.
- Populasi yang terlalu kecil (misalnya 10) menghasilkan solusi yang kurang optimal karena keragaman genetik terbatas.
- Populasi yang optimal (sekitar 60-100) memberikan keseimbangan antara keragaman genetik dan waktu komputasi.

- Algoritma genetika mampu menemukan solusi yang baik melalui proses crossover dan mutasi yang mensimulasikan evolusi alami.

Pemilihan Objective Function

- Ketiga objective function (student conflicts, room conflicts, dan capacity) mengukur aspek kualitas jadwal yang berbeda dan saling melengkapi.
- Pemilihan objective function yang tepat bergantung pada prioritas institusi: kepuasan mahasiswa, efisiensi penggunaan ruangan, atau kepatuhan terhadap kapasitas.

3.2. Saran

Beberapa saran untuk pengembangan lebih lanjut dari penelitian ini adalah:

Optimasi Parameter

- Melakukan grid search atau hyperparameter tuning untuk menemukan kombinasi parameter optimal untuk setiap algoritma.
- Mengimplementasikan adaptive parameter yang menyesuaikan nilai parameter secara dinamis selama proses pencarian.

Hybrid Algorithm

- Mengembangkan algoritma hybrid yang menggabungkan kekuatan dari berbagai algoritma, misalnya Genetic Algorithm dengan Simulated Annealing atau Hill Climbing dengan Tabu Search.
- Mengimplementasikan local search intensif pada solusi-solusi terbaik yang dihasilkan oleh Genetic Algorithm.

Perbaikan Objective Function

- Menambahkan bobot yang dapat dikustomisasi untuk setiap komponen penalti agar institusi dapat menyesuaikan dengan kebutuhan spesifik mereka.
- Menambahkan kriteria tambahan seperti preferensi dosen, distribusi jadwal yang merata sepanjang minggu, dan penghindaran jadwal di jam-jam tertentu.

Peningkatan Representasi Solusi

- Mengimplementasikan constraint handling yang lebih baik untuk menghindari solusi yang tidak feasible.
- Menambahkan mekanisme repair untuk memperbaiki solusi yang melanggar hard constraint.

Eksperimen Lebih Lanjut

- Melakukan eksperimen dengan dataset yang lebih besar dan kompleks untuk menguji skalabilitas algoritma.
- Membandingkan performa algoritma pada berbagai jenis objective function dan menganalisis trade-off yang terjadi.
- Mengimplementasikan visualisasi interaktif untuk memudahkan analisis dan evaluasi hasil penjadwalan.

Implementasi Praktis

- Mengembangkan user interface yang user-friendly untuk memudahkan penggunaan sistem oleh pihak akademik.
- Menambahkan fitur import/export dengan format data yang umum digunakan di institusi pendidikan.
- Mengimplementasikan sistem rekomendasi yang dapat memberikan saran perbaikan jadwal berdasarkan hasil analisis konflik.

4. Pembagian Tugas

Algoritma	NIM
Genetic Algorithm, Output	13523003
Hill Climbing, Objective Function	13523029
Simulated Annealing, Parser	13523033

5. Lampiran

5.1. Referensi

- Slide Kuliah IF3170 “Local Search”
- Slide Kuliah IF3170 “Hill Climbing”
- Slide Kuliah IF3170 “Simulated Annealing”
- Slide Kuliah IF3170 “Genetic Algorithm”

5.2. Link Repository

<https://github.com/bry-ho/TugasBesar1-AI-santAI>